

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

VU MASTERY: YOUR ACADEMIC EDGE



VU MASTERY: YOUR ACADEMIC EDGE



Providing Expert LMS Handling Service |
Achieving 100% Guaranteed Results for
Virtual University Students.

UNLOCK YOUR FULL POTENTIAL.



COMPLETE LMS SOLUTIONS

- ✓ GDB (Graded Discussion Board)
- ✓ Assignments
- ✓ Quizzes
- ✓ Files & Projects
- ✓ All Subjects Covered

WHY CHOOSE US?

- **Unmatched Expertise:**
Decades of combined experience.
- **Confidentiality & Privacy:**
Secure and private service.

EXCELLENCE GUARANTEED

100%

We are committed to
delivering **100% Guaranteed Results.**
Achieve your desired grades
with our support.

- ✓ **Time Management:**
Free up your schedule.
- ✓ **Stress Reduction:**
Let us handle the pressure.
- ✓ **Consistent Updates:**
Track your progress.

GET STARTED TODAY

**For More Information,
Contact Us Now:**

Contact Us

WhatsApp +92 370 5482939

WhatsApp +92 370 5482939

www.vumastery.com

**Don't let LMS stress hold
you back. Elevate your
studies with VU Mastery.**

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Lesson 19 – Random Access Files (Short Summarized Notes)

Sequential Access Files Review

Sequential access files are simple character files in which data is read and written in sequence. The `open()` function is used to open files and `close()` is used to close them. `ifstream` is generally used for reading, while `ofstream` is used for writing. Files can also be opened in append mode (`ios::app`) to add data without deleting existing contents.

Random Access Files

Random access files allow data to be read or written at any location in a file without processing all previous data. This provides faster access to information and makes file handling more efficient.

Position in a File

Every file has a file pointer that indicates the current location inside the file. The `tellg()` function returns the current reading position, while `tellp()` returns the current writing position. Both functions return the position as a long integer.

Setting File Position

The `seekg()` function is used to move the reading pointer, while `seekp()` moves the writing pointer. Positions can be changed relative to the beginning (`ios::beg`), current position (`ios::cur`), or end (`ios::end`) of the file. Positive values move forward and negative values move backward.

Finding File Length

The actual data length of a file can be determined by moving the file pointer to the end using `seekg(0, ios::end)` and then calling `tellg()`. The returned value represents the total number of bytes stored in the file.

Data Insertion in the Middle of a File

Direct insertion of larger data in the middle of a sequential file can corrupt the file structure because new data may overwrite existing data. Traditionally, a new file is created, data is copied up to the insertion point, new data is added, and the remaining data is copied afterward.

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Fixed-Length Records

Random access files commonly use fixed-size records. Each record occupies the same amount of space, allowing direct calculation of record locations. This makes updates and retrieval much faster and prevents file corruption during modifications.

Using Keys for Record Access

A unique key such as an ID number is stored with each record. Instead of searching through the entire file, the key helps quickly locate the desired record. Sorted records with fixed sizes enable direct access to any record.

Reading and Writing Efficiently

Reading or writing one character at a time is slow. The `read()` and `write()` functions allow processing large blocks of data at once, significantly improving performance and reducing disk access operations.

Copying a File in Reverse Order

A file can be copied in reverse by moving the file pointer to the end and reading characters backward using `seekg()`. The last character of the input file becomes the first character of the output file.

Binary File Operations

The `read()` and `write()` functions work in binary mode. Binary files store data in the computer's internal format rather than human-readable text. Functions like `sizeof()` are often used to determine the exact number of bytes to read or write.

Important Exam Points

Random access files allow direct access to any location in a file.

`tellg()` returns the current reading position and `tellp()` returns the current writing position.

`seekg()` is used for moving the input pointer and `seekp()` is used for moving the output pointer.

`ios::beg`, `ios::cur`, and `ios::end` specify movement relative to the beginning, current position, and end of a file.

The combination of `seekg(0, ios::end)` and `tellg()` is used to determine file length.

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Fixed-length records are essential for efficient random access and file updates.

Unique keys such as ID numbers help locate records quickly.

read() and write() are faster than character-by-character operations because they process data in blocks.

Binary files store data in internal machine format rather than readable text.

fstream can be used for both input and output operations by combining ios::in and ios::out.

Important MCQs – Lesson 19 (Random Access Files)

1. Which function is used to determine the current reading position in a file?

- A) seekg()
- B) tellg() ✓
- C) tellp()
- D) write()

2. Which function is used to move the file pointer for reading operations?

- A) seekp()
- B) tellg()
- C) seekg() ✓
- D) getline()

3. Which combination of functions is commonly used to determine the actual length of a file?

- A) open() and close()
- B) read() and write()
- C) seekg() and tellg() ✓
- D) seekp() and tellp()

4. Why are fixed-length records used in random access files?

- A) To reduce file size
- B) To make files readable in Notepad

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

- C) To allow direct access and easy updating of records ✓
- D) To prevent file creation

5. Which functions provide the most efficient method for reading and writing large amounts of file data?

- A) get() and put()
- B) getline() and put()
- C) read() and write() ✓
- D) tellg() and tellp()

Lesson 20 – Structures and Unions (Short Summarized Notes)

Structures

A structure is a user-defined data type that groups related variables under a single name. These variables, called data members, can have different data types. Structures help organize and manage related information as one unit, such as student records, employee data, or car specifications.

Declaration of a Structure

Structures are declared using the struct keyword. The data members are defined inside curly braces. Once declared, the structure becomes a new data type and variables of that type can be created.

```
struct student {  
    char name[60];  
    char address[100];  
    float GPA;  
};
```

Structure Variables

After defining a structure, variables of that type can be declared just like ordinary data types.

```
student std1, std2;
```

Structures can also contain other structures and pointers as data members.

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Initializing Structures

Structure variables can be initialized at the time of declaration by providing values in the same order as the data members.

```
student s1 = {"Ali", "CS201", 19, 2002};
```

This assigns values directly to all data members of the structure.

Accessing Structure Members

The dot operator (.) is used to access data members of a structure variable.

```
s1.age = 20;  
cout << s1.name;
```

Each data member can be accessed individually using the structure variable name followed by the dot operator.

Functions and Structures

Structures can be passed to functions as arguments and can also be returned from functions. When passed by value, a complete copy of the structure is created. Structures may also be passed by reference using pointers to save memory.

Pointers to Structures

A pointer can point to a structure variable. Structure members can be accessed through a pointer using the arrow operator (->).

```
student *sptr;  
sptr = &s1;  
sptr->name;
```

The expression (*sptr).name and sptr->name produce the same result.

Arrays of Structures

Structures can be stored in arrays to manage multiple records efficiently.

```
student s[100];
```

Each element of the array is a complete structure, and members are accessed using array indexing and the dot operator.

```
s[0].name;
```

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Sizeof Operator with Structures

The sizeof() operator can determine the total memory occupied by a structure, including all of its data members.

```
sizeof(s1);
```

This is useful when writing structures to files using binary operations.

Operations on Structures

Structures support assignment between variables of the same structure type.

```
s2 = s1;
```

All data members are copied automatically from one structure variable to another.

Unions

A union is a user-defined data type similar to a structure, but all data members share the same memory location. Only one member can effectively store a value at a time.

```
union intOrChar {  
    int i;  
    char c;  
};
```

Memory Allocation in Unions

The size of a union is equal to the size of its largest data member. Since all members share the same memory area, changing one member affects the interpretation of the others.

Accessing Union Members

Union members are accessed using the dot operator, just like structures.

```
u1.i = 10;  
cout << u1.i;
```

Writing to one member and reading from another may produce unexpected results because both use the same memory location.

Important Exam Points

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

A structure is a collection of variables of different data types grouped under one name.

Structures are declared using the struct keyword.

The variables inside a structure are called data members.

The dot operator (.) is used to access structure members.

The arrow operator (->) is used to access members through a structure pointer.

Structures can contain arrays, pointers, and other structures.

Structures can be passed to and returned from functions.

Arrays of structures are used to store multiple records of the same type.

The sizeof() operator determines the memory occupied by a structure.

A union shares the same memory location among all its data members.

The size of a union equals the size of its largest data member.

Structures allocate separate memory for each member, while unions share a common memory area.

Important MCQs – Lesson 20 (Structures and Unions)

1. Which keyword is used to define a structure in C/C++?

- A) class
- B) union
- C) struct ✓
- D) define

2. Which operator is used to access a data member of a structure variable?

- A) ->
- B) . ✓
- C) ::
- D) *

3. Which operator is used to access structure members through a pointer?

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

- A) .
- B) ::
- C) -> ✓
- D) &

4. What is the size of a union equal to?

- A) Sum of all data members
- B) Average size of members
- C) Size of the smallest member
- D) Size of the largest data member ✓

5. Which statement correctly describes the difference between structures and unions?

- A) Structures and unions allocate memory in the same way.
- B) Structures allocate separate memory for each member, while unions share the same memory location. ✓
- C) Unions allocate separate memory for each member, while structures share the same memory location.
- D) Neither structures nor unions can contain different data types.

Lesson 21 – Bit Manipulation (Short Summarized Notes)

Bit Manipulation

A bit is the smallest unit of memory and can have a value of either 0 or 1. Eight bits make one byte. Bit manipulation means performing operations directly on individual bits instead of entire bytes. It allows efficient memory usage and is widely used in operating systems, file attributes, and low-level programming.

Bit Manipulation Operators

C++ provides special operators for working with bits.

Operator	Purpose
&	Bitwise AND
^	Bitwise XOR

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Operator	Purpose
$\wedge$	Bitwise Exclusive OR (XOR)
$\sim$	Bitwise NOT
$\ll$	Left Shift
$\gg$	Right Shift

These operators work on bits rather than logical expressions.

Bitwise AND Operator (&)

The AND operator compares corresponding bits of two numbers. It returns 1 only when both bits are 1; otherwise it returns 0.

Example:

$$12 \& 8 = 8$$

AND is commonly used to check whether a specific bit is set (1) or not.

Checking a Specific Bit

A bit can be tested by performing AND with a number that contains 1 only at the desired bit position.

`if(number & 0x8)`

A non-zero result means the bit is set, while a zero result means the bit is not set.

Bitwise OR Operator (|)

The OR operator returns 1 if at least one of the compared bits is 1. It returns 0 only when both bits are 0.

Example:

$$8 | 1 = 9$$

OR is mainly used to set a specific bit to 1.

Bitwise Exclusive OR Operator (^)

The XOR operator returns 1 when the two bits are different and 0 when they are the same.

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Example:

$$8 \wedge 1 = 9$$

An important property of XOR is:

$$(a \wedge b) \wedge b = a$$

This property makes XOR useful for encryption, decryption, and swapping values.

Bitwise NOT Operator (~)

The NOT operator is a unary operator that reverses all bits of a number. Every 1 becomes 0 and every 0 becomes 1.

Example:

$$\sim 8$$

If 8 is represented as 1000, then ~ 8 becomes 0111.

Bit Flags

Bit flags are used to represent multiple states or attributes using individual bits. A bit value of 1 indicates that an attribute is enabled, while 0 indicates it is disabled.

Examples include:

Read-only files

Archive files

Read, Write, and Execute permissions in operating systems

Masking

Masking is a technique used to examine, set, clear, or modify selected bits of a number. It is commonly performed using AND, OR, and XOR operators.

Masking is widely used in operating systems, device control, and security systems.

XOR and Password Encryption

The XOR operator can encrypt and decrypt passwords.

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

A password is XORed with a secret number before storage. To retrieve the original password, XOR is applied again using the same secret number.

encrypted = password ^ key;
decrypted = encrypted ^ key;

This provides a simple method of data protection.

XOR for Swapping Numbers

Two numbers can be swapped without using a third variable.

```
a = a ^ b;  
b = b ^ a;  
a = a ^ b;
```

After these operations, the values of a and b are exchanged.

Unsigned Integers

Bit manipulation is usually performed on unsigned integers because signed integers use the most significant bit as the sign bit.

Declaration:

```
unsigned int i, j, k;
```

Unsigned integers store only positive values.

Shift Operators

Shift operators move bits left or right within a number.

Left Shift Operator (<<)

The left shift operator moves bits to the left and inserts zeros on the right side.

Each left shift multiplies the number by 2.

Example:

```
12 << 1 = 24
```

Right Shift Operator (>>)

The right shift operator moves bits to the right and inserts zeros on the left side.

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Each right shift divides the number by 2.

Example:

$$12 \gg 1 = 6$$

Shift operators are faster and more efficient than multiplication and division.

Important Exam Points

Bit is the smallest unit of memory and eight bits make one byte.

Bit manipulation allows direct operations on individual bits.

& is the Bitwise AND operator and is used to check whether a bit is set.

| is the Bitwise OR operator and is used to set a specific bit.

^ is the Exclusive OR (XOR) operator and is used in encryption, decryption, and swapping values.

~ is the NOT operator and reverses all bits of a number.

Bit flags are used to represent multiple attributes using individual bits.

Masking is a technique for testing or modifying selected bits.

Unsigned integers are preferred for bit manipulation.

<< shifts bits to the left and multiplies the number by 2.

>> shifts bits to the right and divides the number by 2.

The XOR property $(a \oplus b) \oplus b = a$ is important for encryption and decryption.

Shift operators are more efficient than normal multiplication and division operations.

Important MCQs – Lesson 21 (Bit Manipulation)

1. Which bit manipulation operator is used to check whether a specific bit is set (1) or not?

- A) |
- B) ^
- C) & ✓
- D) ~

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

2. Which bitwise operator is commonly used to set a specific bit to 1?

- A) &
- B) | ✓
- C) ~
- D) >>

3. Which operator is widely used for encryption, decryption, and swapping two numbers without a temporary variable?

- A) &
- B) |
- C) ^ ✓
- D) ~

4. What is the result of the expression $12 \gg 1$?

- A) 24
- B) 12
- C) 3
- D) 6 ✓

5. Which statement about shift operators is correct?

- A) << divides a number by 2.
- B) >> multiplies a number by 2.
- C) << multiplies a number by 2 and >> divides a number by 2. ✓
- D) Both operators perform logical comparisons only.

Lesson 22 – Revision & Review (Short Summarized Notes)

Bitwise Manipulation and Assignment Operators

Bitwise operators also have compound assignment forms similar to arithmetic operators.

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Examples:

```
a &= b;  
a |= b;  
a ^= b;
```

These are shorthand forms of:

```
a = a & b;  
a = a | b;  
a = a ^ b;
```

The NOT operator (~) is unary and does not have a compound assignment form.

Design Recipe

Programming problems are usually real-world problems that must be analyzed before coding.

The design process includes understanding the problem, simplifying it, creating examples, designing a solution, coding, testing, and reviewing the results.

Attention to detail is extremely important because computers execute instructions exactly as given.

Variables

Variables are symbolic names used to store data in memory. Their values can change during program execution.

Examples:

```
int i;  
double interest;
```

Variables make programs easier to understand and manage than using memory addresses directly.

Data Types

Different data types are used to store different kinds of information.

int stores whole numbers.

short and long are variations of integer types.

unsigned stores only non-negative numbers.

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

float stores real numbers.

double stores larger real numbers with higher precision.

char stores a single character and occupies one byte.

Arrays are collections of variables of the same data type.

Arithmetic Operators

Arithmetic operators perform mathematical calculations.

Operator	Purpose
+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Remainder (Modulus)

Compound arithmetic operators include:

`+= -= *= /= %=`

The modulus operator % is often used to determine whether a number is even or odd.

Logical Operators

Logical operators produce either true or false.

Operator	Purpose
&&	Logical AND
	Logical OR
==	Equal to
!=	Not Equal to
<	Less than
<=	Less than or equal
>	Greater than
>=	Greater than or equal

Do not confuse = (assignment) with == (comparison).

Bitwise Operators

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Bitwise operators work directly on bits.

Operator	Purpose
&	Bitwise AND
	Bitwise OR
^	Bitwise XOR
~	Bitwise NOT

The operators &, |, and ^ are binary operators, while ~ is a unary operator.

Programming Constructs

Programming is based on three fundamental constructs.

Sequential execution executes statements one after another.

Decision structures choose between alternatives.

Loop structures repeat statements multiple times.

These constructs form the foundation of all programs.

Decision Structures

Decision structures allow programs to choose actions based on conditions.

If Statement

The if statement executes code when a condition is true.

```
if(condition)
{
    // statements
}
```

The else part executes when the condition is false.

```
if(condition)
{
}
else
{
}
```

Using braces improves readability and reduces errors.

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Nested If Statements

Nested if statements are used when additional decisions are required inside another if statement.

```
if(a > b)
{
    if(a == 100)
    {
    }
}
```

Logical operators such as && and || are often used in complex conditions.

Loops

Loops repeat a set of statements until a condition becomes false.

While Loop

The condition is checked before execution.

```
while(condition)
{
}
```

The loop may execute zero or more times.

Do-While Loop

The code block executes first and the condition is checked afterward.

```
do
{
}
while(condition);
```

The loop executes at least once.

For Loop

The for loop combines initialization, condition checking, and increment/decrement in one statement.

```
for(int i=0; i<10; i++)
{
}
```

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

It is commonly used when the number of repetitions is known.

Switch, Break and Continue Statements

Switch Statement

The switch statement is used for multi-way selection.

```
switch(choice)
{
    case 1:
        break;
}
```

Break Statement

The break statement immediately terminates a switch or loop.

Continue Statement

The continue statement skips the remaining loop statements and starts the next iteration.

Functions

Functions divide a large program into smaller, manageable modules.

Benefits include:

Code reusability

Easy debugging

Better organization

Improved readability

Function Calling

Call by Value

A copy of the variable is passed to the function.

Changes made inside the function do not affect the original variable.

Call by Reference

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

The address of the variable is passed.

Changes made inside the function affect the original variable.

Pointers are used to implement call by reference.

Top-Down Methodology

A complex problem is divided into smaller sub-problems.

Each sub-problem is implemented as a separate function.

This approach simplifies program development and maintenance.

Arrays

Arrays store multiple values of the same data type under one name.

Example:

```
int age[10];
```

Array indexing starts from 0.

For an array of size 10:

```
age[0] // First element  
age[9] // Last element
```

Arrays can be one-dimensional or multi-dimensional.

Loops are commonly used to process array elements.

Pointers

Pointers store memory addresses instead of actual values.

Example:

```
int *ptr;
```

Pointers are useful for:

Call by reference

Dynamic memory access

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Efficient data manipulation

Array processing

Array names themselves act as constant pointers to the first element.

File Input/Output (File I/O)

Files allow permanent storage of data.

Two common file types are:

Sequential files

Random access files

Important file functions include:

seekg()
seekp()
tellg()
tellp()

seekg() and seekp() move file pointers.

tellg() and tellp() return current file positions.

Important Exam Points

Compound bitwise assignment operators are $\&=$, $|=$, and $\wedge=$.

Variables are symbolic names used to store data in memory.

int, float, double, and char are basic data types.

The modulus operator % returns the remainder after division.

Do not confuse assignment operator = with comparison operator ==.

The three programming constructs are sequence, decision, and loop.

The if statement is used for decision making.

Nested if statements handle complex conditions.

The while loop may execute zero times, while the do-while loop executes at least once.

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

The for loop combines initialization, condition checking, and updating.

break exits a loop or switch statement immediately.

continue skips the remaining loop body and starts the next iteration.

Functions support modular programming.

Call by value passes copies of variables, while call by reference passes addresses.

Array indexing in C/C++ starts from 0.

Pointers store memory addresses.

Arrays are passed to functions by reference by default.

seekg() and seekp() move file pointers, while tellg() and tellp() report file positions.

Important MCQs – Lesson 22 (Revision & Review)

1. Which compound assignment operator is equivalent to $a = a \& b$?

- A) $a |= b$
- B) $a ^= b$
- C) $a \&= b$ ✓
- D) $a \sim= b$

2. Which loop is guaranteed to execute at least once?

- A) for loop
- B) while loop
- C) do-while loop ✓
- D) Nested loop

3. In C/C++, array indexing starts from which value?

- A) 1
- B) -1
- C) 0 ✓
- D) Depends on array size

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

4. What is the main purpose of pointers?

- A) To store character data only
- B) To store memory addresses ✓
- C) To perform arithmetic calculations
- D) To create loops

5. Which function is used to determine the current position of the input file pointer?

- A) seekg()
- B) seekp()
- C) tellg() ✓
- D) tellp()

Lesson 23 – Preprocessor and Macros (Short Summarized Notes)

Preprocessor

The preprocessor is a program that runs before the compiler. It modifies the source code according to preprocessor directives and then passes the modified code to the compiler.

Most preprocessor directives begin with the # symbol.

The preprocessor helps make C/C++ programs more flexible, readable, and maintainable.

#include Directive

The #include directive is used to insert the contents of another file into the source code before compilation.

Including Standard Header Files

```
#include <iostream.h>
```

Angle brackets (< >) are used for standard library header files stored in compiler directories.

Including User-Defined Header Files

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

```
#include "myHeaderFile.h"
```

Quotation marks (" ") are used for header files stored in the current working directory.

Header files usually contain function prototypes, constants, macros, and declarations.

Purpose of Header Files

Header files allow programmers to organize code efficiently.

They contain:

Function prototypes

Macro definitions

Constants

Declarations

Including a header file avoids repeatedly writing the same code in multiple source files.

#define Directive

The #define directive is used to create symbolic constants and macros.

Example:

```
#define PI 3.1415926
```

Whenever PI appears in the program, the preprocessor replaces it with 3.1415926 before compilation.

This makes programs easier to maintain and update.

Advantages of Using #define

Constants can be changed at one location only.

Code becomes more readable.

Errors caused by manual replacement are avoided.

The compiler directly sees the replaced value instead of the symbolic name.

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Other Preprocessor Directives

Some commonly used preprocessor directives are:

```
#include  
#define  
#undef  
#ifdef  
#ifndef  
#if  
#else  
#elif  
#endif  
#error  
#line  
#pragma
```

These directives help control compilation and program configuration.

Conditional Compilation

Conditional compilation allows certain parts of code to be compiled only when specific conditions are met.

Example:

```
#ifdef DEBUG  
cout << "Debug Mode";  
#endif
```

The code executes only if the symbol DEBUG is defined.

Conditional compilation is very useful during program debugging.

#undef Directive

The #undef directive removes a previously defined symbol.

Example:

```
#undef PI
```

After undefining, the symbol can no longer be used until it is defined again.

A symbol must be undefined before redefining it.

Macros

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Macros are symbolic replacements performed by the preprocessor.

They are generally of two types:

Simple Macros

Parameterized Macros

Macros improve execution speed because no function call overhead is involved.

Simple Macros

Simple macros replace a symbol with a fixed value.

Example:

```
#define PI 3.1415926
```

Every occurrence of PI is replaced with the defined value.

Parameterized Macros

Parameterized macros accept arguments similar to functions.

Example:

```
#define square(x) ((x) * (x))
```

Usage:

```
square(5)
```

This becomes:

```
((5) * (5))
```

before compilation.

Importance of Parentheses in Macros

Always use parentheses around parameters and expressions.

Incorrect:

```
#define square(x) x * x
```

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Correct:

```
#define square(x) ((x) * (x))
```

Parentheses prevent operator precedence errors and ensure correct calculations.

Macros vs Functions

Macros perform direct code substitution before compilation.

Functions execute during program runtime.

Advantages of Macros

Faster execution

No function call overhead

Useful for simple operations

Advantages of Functions

Smaller executable size

Better readability

Easier debugging

Suitable for complex tasks

Example: Circle Area Macro

A macro can be used to calculate the area of a circle.

```
#define PI 3.14159  
#define CIRCLEAREA(X) (PI * (X) * (X))
```

Usage:

```
CIRCLEAREA(radius)
```

The formula is substituted directly before compilation.

Portability of Header Files

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

C/C++ programs are highly portable because standard header file names remain mostly the same across different operating systems and compilers.

Programs can often be moved from one system to another with little or no modification.

Custom header files also improve portability by keeping reusable code in one place.

Important Exam Points

The preprocessor executes before the compiler.

All preprocessor directives start with the # symbol.

#include is used to include header files.

Angle brackets < > are used for standard header files.

Quotation marks " " are used for user-defined header files.

#define creates symbolic constants and macros.

#undef removes a previously defined symbol.

#ifdef checks whether a symbol is defined.

Conditional compilation is useful for debugging.

Macros are expanded by the preprocessor before compilation.

Parameterized macros can accept arguments.

Always use parentheses in parameterized macros.

Macros execute faster than functions because there is no function call overhead.

Functions improve readability and reduce code size.

A symbol must be undefined before being redefined.

Do not use variable names that start with an underscore (_).

Important MCQs – Lesson 23 (Preprocessor and Macros)

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

1. Which symbol is used at the beginning of all preprocessor directives in C/C++?

- A) \$
 - B) @
 - C) # ✓
 - D) %
-

2. Which directive is used to define a symbolic constant or macro?

- A) #include
 - B) #define ✓
 - C) #ifdef
 - D) #undef
-

3. Which form of the #include directive is used for user-defined header files?

- A) #include <myfile.h>
 - B) #include (myfile.h)
 - C) #include [myfile.h]
 - D) #include "myfile.h" ✓
-

4. Which preprocessor directive is used to remove a previously defined symbol?

- A) #ifdef
 - B) #endif
 - C) #undef ✓
 - D) #pragma
-

5. Why should parentheses always be used in parameterized macros?

- A) To make the program compile faster
 - B) To reduce memory usage
 - C) To avoid operator precedence errors and ensure correct calculations ✓
 - D) To convert macros into functions
-

Lesson 24 – Dynamic Memory Allocation

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Overview

Dynamic Memory Allocation allows memory to be allocated during program execution instead of reserving memory at compile time.

It helps in efficient memory utilization and avoids unnecessary wastage of memory resources.

Memory for dynamic allocation is taken from the **Heap**, while static memory is allocated on the **Stack**.

Memory Allocation

Memory allocation is the process of reserving memory space for variables and data.

Static memory allocation occurs at compile time and its size cannot be changed during program execution.

Dynamic memory allocation occurs at run time and memory size can be adjusted according to program requirements.

Static allocation is simple and safe but may waste memory.

Dynamic allocation is flexible and efficient but requires careful memory management.

Dynamic Memory Allocation

Dynamic memory allocation is used when the required memory size is not known beforehand.

Memory is allocated from the heap during program execution.

The starting address of allocated memory is stored in a pointer.

Pointers are necessary because dynamically allocated memory does not have predefined variable names.

Dynamic allocation improves memory utilization and reduces wastage.

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Void Pointer

A void pointer can point to any data type.

It is declared as:

```
void *ptr;
```

A void pointer cannot be dereferenced directly.

It must be type cast before use.

Example:

```
(int *) ptr;
```

Casting converts the void pointer into a specific data type pointer.

NULL Pointer

NULL is a special pointer value that contains all zero bits.

It indicates that the pointer is not pointing to any valid memory location.

NULL is defined in **stdlib.h** and **stddef.h**.

A pointer should always be checked against NULL before use.

calloc Function

The calloc function allocates memory dynamically from the heap.

Syntax:

```
void *calloc(size_t n, size_t el_size);
```

The first argument specifies the number of elements.

The second argument specifies the size of each element.

Example:

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

```
calloc(1000, sizeof(int));
```

This allocates memory for 1000 integers.

Memory allocated by calloc is automatically initialized to zero.

If allocation fails, calloc returns NULL.

Example:

```
int *iptr;  
iptr = (int *) calloc(1000, sizeof(int));
```

Always check for NULL after allocation.

malloc Function

The malloc function allocates a block of memory from the heap.

Syntax:

```
void *malloc(size_t size);
```

Example:

```
malloc(1000 * sizeof(int));
```

This allocates memory for 1000 integers.

Unlike calloc, memory allocated by malloc is not initialized.

The allocated memory may contain garbage values.

If memory allocation fails, malloc returns NULL.

Example:

```
int *iptr;  
iptr = (int *) malloc(1000 * sizeof(int));
```

free Function

The free function releases dynamically allocated memory.

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Syntax:

```
free(pointer);
```

Example:

```
free(iptr);
```

After freeing memory, it becomes available for other programs.

Failure to free memory can cause memory leaks.

A good programming practice is to set the pointer to NULL after freeing memory.

Example:

```
free(iptr);  
iptr = NULL;
```

realloc Function

The realloc function changes the size of previously allocated memory.

Syntax:

```
realloc(void *ptr, size_t size);
```

Example:

```
realloc(iptr, 25 * sizeof(int));
```

It can increase or decrease memory size.

If sufficient adjacent memory is available, the block is expanded.

Otherwise, a new memory block is allocated and existing data is copied.

If realloc fails, it returns NULL.

Always check the returned pointer after reallocation.

Memory Leak

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

A memory leak occurs when allocated memory is not freed and no pointer remains to access it.

The memory becomes unusable but remains reserved.

Repeated memory leaks reduce available heap memory.

Large memory leaks may slow down or crash a system.

To avoid memory leaks:

Always free allocated memory.

Ensure memory is released before program termination.

Assign NULL to pointers after freeing memory.

Dangling Pointer

A dangling pointer points to memory that has already been freed.

Using a dangling pointer can cause unexpected behavior.

It may corrupt data or crash the program.

Example:

```
free(ptr);
```

After this statement, ptr becomes a dangling pointer if not assigned NULL.

Solution:

```
free(ptr);  
ptr = NULL;
```

Always reset pointers after freeing memory.

Advantages of Dynamic Memory Allocation

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Memory is allocated only when needed.

Memory wastage is minimized.

Program becomes flexible and efficient.

Memory size can be increased or decreased during execution.

Large data structures can be handled easily.

Disadvantages of Dynamic Memory Allocation

Programmer must manage memory manually.

Improper handling can cause memory leaks.

Dangling pointers may occur.

Additional overhead is required for memory management.

Programs may crash if memory is not handled carefully.

Examples of Dynamic Memory Allocation

Memory can be allocated for student records according to actual class strength.

Memory can be allocated for structures such as employee records.

Large files can be loaded into dynamically allocated memory for efficient processing.

Dynamic memory is widely used in file handling, linked lists, trees, stacks, and queues.

Important Exam Points

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Dynamic memory allocation uses the **Heap**, while static memory allocation uses the **Stack**.

The starting address of dynamically allocated memory is stored in a pointer.

A **void pointer** must be type cast before use.

calloc() allocates memory and initializes it to zero.

malloc() allocates memory but does not initialize it.

free() releases allocated memory back to the heap.

realloc() changes the size of previously allocated memory.

Failure of malloc, calloc, or realloc returns **NULL**.

A **Memory Leak** occurs when allocated memory is not freed.

A **Dangling Pointer** points to memory that has already been freed.

Always check for **NULL** immediately after memory allocation.

Always free allocated memory before program termination.

Assign **NULL** to pointers after freeing memory to avoid dangling pointers.

Important MCQs – Lesson 24 (Dynamic Memory Allocation)

1. Which area of memory is used for dynamic memory allocation?

- A) Stack
- B) Register
- C) Heap ✓
- D) Cache

2. Which function allocates memory and automatically initializes it to zero?

- A) malloc()
- B) realloc()
- C) free()
- D) calloc() ✓

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

3. What does the malloc() function return if memory allocation fails?

- A) 0
- B) Garbage Value
- C) NULL ✓
- D) -1

4. Which function is used to release dynamically allocated memory back to the heap?

- A) malloc()
- B) calloc()
- C) realloc()
- D) free() ✓

5. A pointer that points to memory that has already been freed is called a:

- A) Null Pointer
- B) Void Pointer
- C) Dangling Pointer ✓
- D) Wild Pointer

Lesson 25 – Introduction to C++ Features

Lecture Overview

C++ is an extension of the C language that introduces several new programming features. While C follows the structured programming approach, C++ provides additional facilities such as default function arguments, inline functions, flexible variable declarations, and function overloading.

History of C/C++

C language was developed at Bell Labs in the 1970s.

Before C, programmers used Machine Language and Assembly Language.

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

High-level languages such as COBOL and FORTRAN were later introduced.

Early programs often contained spaghetti code, which was difficult to read and maintain.

Structured programming was introduced to solve these problems by dividing large problems into smaller modules.

In the 1980s, **Bjarne Stroustrup** enhanced C and developed C++, originally called **C with Classes**.

Java later evolved from many of the ideas introduced in C++.

Structured Programming

Structured programming follows the **top-down approach**.

A large problem is divided into smaller functions.

Each function performs a specific task.

Programs become easier to understand, debug, and maintain.

Rules of Structured Programming

Keep functions short and focused on one task.

Follow the **Single Entry Single Exit** principle whenever possible.

Write meaningful comments in programs.

Document function purpose, parameters, and return values.

Reuse existing code whenever possible.

Limitations of Structured Programming

Structured programming focuses mainly on functions.

Data is treated separately from functions.

For complex systems, managing data becomes difficult.

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Programs should be organized around data as well as functions.

This need led to **Object Oriented Programming (OOP)**.

OOP focuses on data and the operations performed on that data.

C++ was developed to overcome the limitations of structured programming.

Default Function Arguments

Default arguments allow parameters to have predefined values.

If the caller does not provide a value, the default value is used.

Default values are specified in the function prototype or definition.

Example

```
void f(int i = 1, double x = 10.5);
```

Function call:

```
f();
```

Output:

```
i = 1  
x = 10.5
```

Rules of Default Arguments

Arguments are assigned from left to right.

Parameters with default values must be placed on the right side.

A parameter on the left cannot be skipped.

Default arguments reduce repeated code and simplify function calls.

Placement of Variable Declarations

In C, variables are usually declared at the beginning of a function.

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

C++ allows variables to be declared anywhere in the code.

Variables should preferably be declared close to where they are used.

This improves readability and maintainability.

Scope of Variables

A variable is accessible only within the block where it is declared.

After leaving the block, the variable is destroyed.

Variables declared inside loops remain valid only within their scope.

Inline Functions

Inline functions are small functions whose code is inserted directly at the point of function call.

The keyword **inline** is used before the function definition.

Syntax

```
inline int max(int a, int b)
{
    return (a > b) ? a : b;
}
```

Advantages of Inline Functions

Reduce function call overhead.

Improve execution speed for small functions.

Provide automatic type checking.

Avoid side effects common in macros.

Disadvantages of Inline Functions

Increase program size if used many times.

Compiler may ignore the inline request.

Large functions are generally not expanded inline.

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Inline Functions vs Macros

Macros are expanded by the preprocessor before compilation.

Macros do not perform type checking.

Macros may produce unexpected results because of side effects.

Inline functions behave like normal functions and perform type checking.

Inline functions are safer and more reliable than macros.

Macro Example

```
#define square(x) (x) * (x)
```

Inline Function Example

```
inline int square(int x)
{
    return x * x;
}
```

Inline functions are generally preferred over macros.

Function Overloading

Function overloading allows multiple functions to have the same name.

The compiler differentiates them using the number or type of parameters.

This improves program readability and usability.

Example

```
int sqrt(int i);
double sqrt(double d);
```

The compiler automatically selects the correct version according to the argument type.

Rules of Function Overloading

Function names remain the same.

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Parameter types or number of parameters must differ.

Return type alone cannot distinguish overloaded functions.

Invalid Overloading

```
int f(int);  
double f(int);
```

This produces an error because only return types differ.

Advantages of Function Overloading

Improves code readability.

Reduces the need to remember multiple function names.

Allows the same operation to work with different data types.

Makes programs easier to maintain.

Name Mangling

The compiler internally generates unique names for overloaded functions.

This process is called **Name Mangling**.

Name mangling helps the compiler distinguish overloaded functions.

Important Exam Points

C language was developed at Bell Labs in the 1970s.

Bjarne Stroustrup developed C++ in the early 1980s.

Structured programming follows the top-down approach.

Default arguments must be placed on the right side of parameter lists.

Arguments are assigned to parameters from left to right.

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

C++ allows variable declarations anywhere in a function.

Variables are only accessible within their scope.

Inline functions reduce function call overhead.

Inline functions provide type checking while macros do not.

Macros can cause side effects whereas inline functions are safer.

Function overloading allows multiple functions with the same name.

Overloaded functions are distinguished by parameter type and number.

Return type alone cannot be used for function overloading.

Name mangling is used internally by the compiler to identify overloaded functions.

MCQ 1

Default arguments in C++ should always be placed:

- A) At the beginning of the parameter list
- B) In the middle of the parameter list
- C) Anywhere in the parameter list
- D) **At the right end of the parameter list ✓**

MCQ 2

Which feature of C++ allows multiple functions to have the same name but different parameter lists?

- A) Inline Functions
- B) Default Arguments
- C) **Function Overloading ✓**
- D) Name Mangling

MCQ 3

What is the major limitation of Structured Programming that led to the development of Object-Oriented Programming?

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

- A) Programs execute slowly
 - B) **It focuses on functions rather than data** ✓
 - C) It does not support loops
 - D) It does not support functions
-

MCQ 4

Which statement about Inline Functions is correct?

- A) They do not perform type checking
 - B) They are expanded by the preprocessor
 - C) **They reduce function call overhead and provide type checking** ✓
 - D) They can only be used with classes
-

MCQ 5

Which of the following is NOT a valid way to overload a function?

- A) Changing the number of parameters
 - B) Changing the type of parameters
 - C) Changing both number and type of parameters
 - D) **Changing only the return type** ✓
-

Lesson 26 – Classes and Objects

Classes and Objects

A class is a user-defined data type that contains both data members and member functions.

Data members store information while member functions manipulate that information.

Member functions are also called methods.

An object is an instance of a class.

When a class is created, it becomes a new data type just like int or char.

Objects reserve memory for the data members of the class.

A class groups related data and functions into a single unit.

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Definition of a Class

A class is declared using the **class** keyword.

General syntax:

```
class ClassName
{
    // data members
    // member functions
};
```

Example:

```
class Date
{
    int day;
    int month;
    int year;
};
```

Object creation:

Date myDate;

Here, **myDate** is an object of the class **Date**.

Separation of Interface from Implementation

One of the most important concepts of Object-Oriented Programming is the separation of interface from implementation.

The interface contains only those functions that users can access.

The implementation contains the internal details and data of the class.

Users interact with the class through public functions without knowing how the class works internally.

This concept provides data security and flexibility.

The internal implementation can be changed without affecting programs that use the class.

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Examples from real life include wristwatches and automobiles where users only see the controls, not the internal mechanism.

Private and Public Access Specifiers

Class members are private by default.

Private members cannot be accessed directly from outside the class.

Public members can be accessed from outside the class.

Syntax:

```
class Date
{
private:
    int day;
    int month;
    int year;

public:
    void setDay(int);
    void display();
};
```

Private data is hidden from users.

Public functions act as an interface to access and manipulate private data.

This concept is known as **Encapsulation**.

Structure of a Class

A class generally contains two sections.

The private section contains data members and hidden functions.

The public section contains member functions that can be called by users.

Example:

```
class Date
{
private:
    int day;
```

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

```
int month;  
int year;  
  
public:  
    void setDay(int);  
    void setMonth(int);  
    void setYear(int);  
    void display();  
};
```

Normally data members are kept private and member functions are kept public.

Member Functions

Member functions operate on the data members of the class.

They can access both private and public data members.

Functions are called using the object name and dot operator.

Example:

```
myDate.setMonth(10);  
myDate.display();
```

Functions are defined outside the class using the scope resolution operator (::).

Example:

```
void Date::setMonth(int i)  
{  
    month = i;  
}
```

The double colon (::) is called the scope resolution operator.

It identifies the class to which the function belongs.

Sample Date Class

The Date class contains three private data members:

day

month

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

year

Public member functions include:

setDay()

setMonth()

setYear()

display()

These functions allow controlled access to the private data.

Objects can call these functions using the dot operator.

Different objects maintain separate copies of data members.

Constructor

A constructor is a special member function of a class.

It has the same name as the class.

A constructor has no return type.

It is automatically called when an object is created.

The main purpose of a constructor is to initialize an object.

Example:

```
class Date
{
public:
    Date(int,int,int);
};
```

Definition:

```
Date::Date(int d,int m,int y)
{
    day = d;
    month = m;
    year = y;
}
```

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Object creation:

```
Date myDate(1,1,2003);
```

The object is created and initialized simultaneously.

Default Constructor

A constructor with no arguments is called a default constructor.

Example:

```
Date::Date()  
{  
    day = 1;  
    month = 1;  
    year = 1900;  
}
```

If no constructor is written, C++ automatically provides a default constructor.

The default constructor initializes the object to a valid starting state.

Default Arguments with Constructors

Constructors can also use default arguments.

Example:

```
Date::Date(int d,int m,int y = 2002)  
{  
    day = d;  
    month = m;  
    year = y;  
}
```

Object creation:

```
Date date1(1,1,2000);
```

Year becomes 2000.

```
Date date2(1,1);
```

Year takes the default value 2002.

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Default arguments make object creation easier and more flexible.

Constructor Overloading

A class can have multiple constructors.

This is called constructor overloading.

Different constructors can accept different numbers or types of arguments.

Example:

```
Date();  
Date(int,int,int);  
Date(char *);
```

The compiler selects the appropriate constructor according to the arguments passed.

Constructor overloading improves usability of classes.

Getter and Setter Functions

Setter functions assign values to private data members.

Examples:

```
setDay()  
setMonth()  
setYear()
```

Getter functions return values of private data members.

Examples:

```
getDay()  
getMonth()  
getYear()
```

These functions provide controlled access to class data.

They help maintain data security and validity.

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Scope Resolution Operator (::)

The scope resolution operator connects a function definition with its class.

Syntax:

```
ReturnType ClassName::FunctionName()
{
}
```

Example:

```
void Date::display()
{
    cout << day;
}
```

Without scope resolution, the compiler cannot determine which class the function belongs to.

Important Exam Points

A class is a user-defined data type containing data members and member functions.

Objects are instances of a class.

Member functions are also called methods.

Class members are private by default.

Private members cannot be accessed directly outside the class.

Public members form the interface of the class.

Encapsulation means hiding data and providing controlled access through functions.

The dot (.) operator is used to access public members of an object.

The scope resolution operator (::) is used to define member functions outside the class.

A constructor has the same name as the class.

Constructors have no return type.

Constructors are automatically called when objects are created.

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

The default constructor takes no arguments.

Constructors are mainly used to initialize objects.

Constructors can be overloaded.

Default arguments can be used with constructors.

Getter functions return data while setter functions modify data.

Separating interface from implementation is a fundamental concept of Object-Oriented Programming.

1. Which access specifier is the default for class members in C++?

- A) Public
 - B) Protected
 - C) Private ✓
 - D) Global
-

2. What is an object in C++?

- A) A function of a class
 - B) An instance of a class ✓
 - C) A data member
 - D) A constructor
-

3. Which operator is used to define a member function outside a class?

- A) .
 - B) ->
 - C) :: ✓
 - D) &&
-

4. What is the primary purpose of a constructor?

- A) Delete an object
 - B) Display data
 - C) Initialize an object ✓
 - D) Create a function
-

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

5. Which statement about constructors is correct?

- A) Constructors have a return type
- B) Constructors can have any name
- C) Constructors have the same name as the class ✓
- D) Constructors are called manually only

6. Which concept allows hiding data and providing controlled access through public functions?

- A) Inheritance
- B) Encapsulation ✓
- C) Polymorphism
- D) Recursion

Lesson 27 – Classes, Constructors, Utility Functions and Destructors

Classes and Objects

Classes and Objects are the foundation of Object-Oriented Programming (OOP). OOP focuses on objects that combine data and behavior instead of only functions.

A class is a user-defined data type that contains data members and member functions. It works as a blueprint for creating objects.

Objects are real instances of a class and occupy memory when created.

Objects represent real-world entities such as students, cars, or bank accounts. Their properties are stored in data members and actions are handled by member functions.

Class Structure and Data Hiding

A class is divided into two main parts.

The private section contains data members and cannot be accessed directly from outside the class.

The public section contains member functions that provide controlled access to private data.

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

This concept is called encapsulation or data hiding.

Example of Class Structure

```
class Date {  
private:  
    int day;  
    int month;  
    int year;  
  
public:  
    void setDate(int d, int m, int y);  
    void displayDate();  
};
```

Constructors

A constructor is a special member function that is automatically called when an object is created.

It has the same name as the class and has no return type.

It is used to initialize data members.

Example of Constructor

```
class Date {  
private:  
    int day, month, year;  
  
public:  
    Date() {  
        day = 1;  
        month = 1;  
        year = 2000;  
    }  
};
```

Types of Constructors

Compiler Generated Constructor

If no constructor is defined, the compiler creates one automatically.

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

It does not initialize values properly.

Simple Constructor

A constructor with no parameters.

```
Date() {  
    day = 0;  
    month = 0;  
    year = 0;  
}
```

Parameterized Constructor

A constructor that takes values as input.

```
class Date {  
private:  
    int day, month, year;  
  
public:  
    Date(int d, int m, int y) {  
        day = d;  
        month = m;  
        year = y;  
    }  
};
```

Object Creation Using Constructor

```
Date d1(1, 1, 2000);  
Date d2(10, 5, 2025);
```

Constructor Overloading

Multiple constructors in the same class.

```
class Date {  
private:  
    int day, month, year;  
  
public:  
    Date() {  
        day = 1;  
        month = 1;  
    }  
};
```

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

```
    year = 2000;
}

Date(int d, int m) {
    day = d;
    month = m;
    year = 2024;
}

Date(int d, int m, int y) {
    day = d;
    month = m;
    year = y;
}
};
```

Constructor Characteristics

Constructors have the same name as the class.

They have no return type.

They are automatically called.

They can be overloaded.

They can use default arguments.

Utility Functions

Utility functions are helper functions used inside a class.

They are usually private and cannot be accessed from outside the class.

They help reduce code repetition.

Example of Utility Function

```
class Date {
private:
    int day, month, year;

    bool isValid(int d, int m, int y) {
        return (m >= 1 && m <= 12);
    }
}
```

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

```
public:
    void setDate(int d, int m, int y) {
        if (isValid(d, m, y)) {
            day = d;
            month = m;
            year = y;
        }
    }
};
```

Destructors

A destructor is a special function called automatically when an object is destroyed.

It has the same name as the class with a tilde (~).

It has no return type and no parameters.

Example of Destructor

```
class Date {
public:
    Date() {
        // constructor
    }

    ~Date() {
        // destructor
    }
};
```

Purpose of Destructors

Used to free memory.

Used for cleanup operations.

Called automatically when object goes out of scope.

Object Memory Management

Each object has its own copy of data members.

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Member functions are shared among all objects.

Example of Objects

Date d1;
Date d2;
Date d3;

Each object stores separate values.

Calling Member Functions

d1.setDate(1,2,2020);
d2.setDate(5,6,2022);

Advantages of Constructors

Ensure proper initialization.

Prevent garbage values.

Improve program reliability.

Advantages of Destructors

Free memory automatically.

Prevent memory leaks.

Improve system stability.

Summary

A class combines data and functions.

Constructors initialize objects automatically.

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Destructors clean up objects automatically.

Utility functions support internal operations.

Objects store separate data but share functions.

Important Exam Points

A constructor has the same name as the class and no return type.

Constructors are automatically called.

Parameterized constructors take values.

Constructor overloading is allowed.

Utility functions are private helper functions.

A destructor starts with ~ symbol.

Only one destructor is allowed.

Destructors are automatically called.

Each object has its own data members.

All objects share member functions.

Important MCQs – Lesson 27

1. What is a constructor?

- A) A normal function
 - B) A special function called when object is created ✓
 - C) A loop
 - D) A variable
-

2. What is the main purpose of a constructor?

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

A) To delete objects

B) To initialize data members ✓

C) To print output

D) To end program

3. What is a destructor used for?

A) Input values

B) Initialize variables

C) Free memory and cleanup ✓

D) Create objects

4. How many destructors can a class have?

A) Two

B) Many

C) One only ✓

D) Zero

5. What is a utility function?

A) Main function

B) Public function only

C) Helper function inside a class (usually private) ✓

D) External library function

Lesson 28 – Memory Allocation in C++ (new and delete, Classes, Objects)

Lecture Overview

In previous lectures, we studied classes, objects, constructors, and destructors. In this lecture, we extend these concepts to memory allocation.

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

We learn how memory is allocated in C and C++, and how C++ provides a better and safer approach using new and delete operators.

We also compare C-style memory allocation with C++ style memory allocation.

Memory Allocation in C

In C, dynamic memory is allocated using malloc(), calloc(), and realloc().

Memory is allocated at runtime from the heap.

Example structure:

```
struct abc {  
    int integer;  
    float floatingpoint;  
};
```

Creating object:

```
struct abc xyz;
```

Accessing members:

```
xyz.integer = 2134;  
xyz.floatingpoint = 234.34;
```

Pointer to structure:

```
struct abc* abcPtr;  
abcPtr = &xyz;  
abcPtr->integer = 2134;  
abcPtr->floatingpoint = 234.34;
```

Memory Allocation in C (Problems)

Memory size must be calculated manually and explicitly.

Type casting is required when using malloc().

malloc() returns a void pointer.

Memory may contain garbage values.

Memory must be manually freed using free().

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Memory Allocation in C++

C++ uses the new operator for dynamic memory allocation.

```
int *p;  
p = new int;
```

There is no need for sizeof, casting, or manual byte calculation.

Memory is allocated from free store.

delete Operator in C++

Memory allocated using new must be released using delete.

```
delete p;
```

For arrays, delete[] is used.

```
int *p = new int[10];  
delete[] p;
```

new Operator and Classes

Objects can also be created dynamically using new.

```
class Date {  
public:  
    int day, month, year;  
};
```

```
Date *dptr;  
dptr = new Date;
```

Working of new Operator

The new operator automatically allocates memory.

It automatically calls the constructor of the class.

It returns a typed pointer so no casting is required.

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Example Program 1 (Memory Leak Case)

```
class MyDate {  
public:  
    MyDate() {  
        cout << "Constructor called";  
    }  
  
    ~MyDate() {  
        cout << "Destructor called";  
    }  
};  
  
int main() {  
    MyDate *dptr;  
    dptr = new MyDate[10];  
}
```

In this program, destructors are not called because delete is missing, which causes a memory leak.

Classes and Structures in C++

In C++, structures can contain functions like classes.

In struct, members are public by default while in class, members are private by default.

It is good practice to explicitly write public and private keywords for clarity.

new Operator and Constructors

When new is used, memory is allocated and constructor is automatically called.

```
Date *dptr;  
dptr = new Date;
```

For arrays:

```
Date *dptr;  
dptr = new Date[10];
```

Constructors are called for each object created in the array.

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

delete Operator and Classes

When delete is used, destructor is automatically called.

```
delete dptr;
```

For arrays:

```
delete[] dptr;
```

This ensures proper cleanup and avoids memory leaks.

Example Program 2 (Proper Memory Management)

```
class MyDate {  
public:  
    MyDate() {  
        cout << "Constructor called";  
    }  
  
    ~MyDate() {  
        cout << "Destructor called";  
    }  
};  
  
int main() {  
    MyDate *dptr;  
    dptr = new MyDate[10];  
  
    delete[] dptr;  
}
```

In this case, destructors are properly called for all objects.

new and delete Outside Constructors/Destructors

new and delete can be used anywhere in a program depending on requirement.

```
int *p = new int;  
delete p;
```

This is useful for dynamic resizing and runtime memory management.

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Class Abstraction

A class hides internal implementation from the user.

Only the interface (functions) is visible to the user.

Internal details remain hidden and protected.

This concept is called abstraction.

Messages and Methods

Calling a function on an object is considered sending a message.

Methods are functions defined inside a class.

Objects respond to these messages by executing methods.

Classes Extend the Language

C++ allows creation of user-defined data types using classes.

This extends the language beyond built-in types.

Complex numbers and other structures can be implemented using classes.

Operators can also be overloaded for custom behavior.

Important Exam Points

In C, memory allocation is done using malloc() while in C++ it is done using new.

The new operator automatically calls constructors.

The delete operator automatically calls destructors.

new does not require sizeof or type casting.

Memory allocated using new must be freed using delete.

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

For arrays, delete[] must be used.

In C++, memory is allocated from free store while in C it is allocated from heap.

If new fails, it returns 0.

If malloc() fails, it returns NULL.

Constructors are automatically called when objects are created using new.

Destructors are automatically called when objects are deleted.

Struct members are public by default while class members are private by default.

new can be used inside constructors for dynamic memory allocation.

delete is usually used in destructors for cleanup.

Memory leak occurs when dynamically allocated memory is not freed.

C++ classes allow extension of the programming language by creating new data types.

Important MCQs – Lesson 28 (Memory Allocation in C++)

1. What does the new operator do in C++?

- A) Only allocates memory
 - B) Allocates memory and calls constructor ✓
 - C) Only calls destructor
 - D) Frees memory
-

2. Which operator is used to free memory allocated by new?

- A) free
 - B) remove
 - C) delete ✓
 - D) clear
-

3. What will happen when this code runs?

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

```
class A {  
public:  
    A() {  
        cout << "Constructor";  
    }  
};  
  
int main() {  
    A *p = new A;  
}
```

- A) Constructor is not called
 - B) Constructor is called ✓
 - C) Destructor is called
 - D) Error
-

4. Which is correct for deleting an array allocated by new?

- A) delete p
 - B) delete[] p ✓
 - C) free(p)
 - D) remove p
-

5. What is a memory leak?

- A) Memory is reused automatically
 - B) Memory is freed properly
 - C) Allocated memory is not freed ✓
 - D) Memory is increased automatically
-

Lecture No. 29 – Friend Functions and Friend Classes (C++)

Friend Functions

Friend functions are non-member functions of a class that are allowed to access private and protected data members of that class. They are used when controlled external access to class data is required without breaking encapsulation completely.

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

A friend function is declared inside the class using the keyword friend, but it is defined outside the class like a normal function.

Declaration of Friend Function

A friend function is declared inside a class as:

```
friend return_type function_name(parameters);
```

It can be placed anywhere in the class, but usually written at the top.

It is defined outside the class without the keyword friend.

Sample Program 1 (Single Class Friend Function)

```
#include <iostream>
using namespace std;

class myClass {
    int topSecret;

public:
    myClass() {
        topSecret = 100;
    }

    void display() {
        cout << "\nValue: " << topSecret;
    }

    friend void increment(myClass *, int);
};

void increment(myClass *a, int i) {
    a->topSecret += i;
}

int main() {
    myClass x;
    x.display();

    increment(&x, 10);
    x.display();
}
```

This program shows that friend function directly accesses private data and modifies it.

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Sample Program 2 (Friend Function with Two Classes)

```
#include <iostream>
using namespace std;

class myClass2;

class myClass1 {
    int topSecret;

public:
    myClass1() { topSecret = 100; }

    friend void addBoth(myClass1, myClass2);
};

class myClass2 {
    int topSecret;

public:
    myClass2() { topSecret = 200; }

    friend void addBoth(myClass1, myClass2);
};

void addBoth(myClass1 a, myClass2 b) {
    cout << "\nClass1: " << a.topSecret;
    cout << "\nClass2: " << b.topSecret;
    cout << "\nSum: " << a.topSecret + b.topSecret;
}

int main() {
    myClass1 a;
    myClass2 b;

    addBoth(a, b);
}
```

This shows a single friend function accessing private data of two different classes.

Sample Program 3 (Multiple Friend Functions)

```
#include <iostream>
using namespace std;

class myClass2;

class myClass1 {
```

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

```
float value;  
  
public:  
    myClass1() { value = 200; }  
  
    friend float addBoth(myClass1, myClass2);  
    friend float subBoth(myClass1, myClass2);  
};  
  
class myClass2 {  
    float value;  
  
public:  
    myClass2() { value = 100; }  
  
    friend float addBoth(myClass1, myClass2);  
    friend float subBoth(myClass1, myClass2);  
};  
  
float addBoth(myClass1 a, myClass2 b) {  
    return a.value + b.value;  
}  
  
float subBoth(myClass1 a, myClass2 b) {  
    return a.value - b.value;  
}
```

This shows multiple friend functions performing different operations.

Friend Classes

A class can be declared as a friend of another class using:

```
friend class ClassName;
```

When a class is a friend, all its member functions can access private data of the other class.

Example of Friend Class

```
class ClassOne {  
    friend class OtherClass;  
  
private:  
    int topSecret;  
};  
  
class OtherClass {
```

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

```
public:  
    void change(ClassOne co) {  
        co.topSecret++;  
    }  
};
```

Key Concept

Friendship is one-way. If Class A declares Class B as friend, Class B can access Class A's private data, but not vice versa unless separately declared.

Limitations

Friendship is not transitive. If A is friend of B and B is friend of C, A is not automatically friend of C.

Friend functions and friend classes reduce encapsulation and should be used carefully.

Important Exam Points

Friend functions are not member functions but can access private data.

They are declared with friend inside class and defined outside.

Friend classes give full access to another class's private data.

Friendship is one-way and non-transitive.

Lesson 29 – Friend Functions and Friend Classes in C++

1. What is a friend function in C++?

- A) A member function of a class
- B) A function that belongs to another class
- C) A **non-member function that can access private data of a class** ✓
- D) A constructor of a class

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

2. Where is a friend function defined in C++?

- A) Inside the class only
 - B) Outside the class without using the keyword friend ✓**
 - C) Inside main function
 - D) Inside private section only
-

3. Which statement about friend functions is TRUE?

- A) They are always member functions
 - B) They cannot access private data of a class
 - C) They are declared using keyword friend inside a class ✓**
 - D) They are automatically created by compiler
-

4. What happens if a non-friend function tries to access private data of a class?

- A) It works normally
 - B) Compiler gives an error ✓**
 - C) It becomes a friend automatically
 - D) Program runs with warning only
-

5. Which of the following is TRUE about friend classes?

- A) Friendship is bidirectional automatically
 - B) Friend class can access private data of another class ✓**
 - C) Friend class cannot access any data
 - D) Friend class is always a member class
-

Lecture No. 30 – Reference Data Type in C++

Reference Data Type

A reference in C++ is an alias or synonym for an existing variable. It provides another name for the same memory location instead of creating a new variable. References are declared using the & symbol in the declaration.

A reference must always be initialized when it is declared. Once initialized, it cannot be changed to refer to another variable.

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Declaration and Working of References

A reference is declared as:

```
int &j = i;
```

Here, j becomes another name for i. Both refer to the same memory location. Any change in one variable will automatically reflect in the other.

Unlike the address-of operator used in expressions, the & in declaration means reference creation.

Example 1 – Basic Reference Usage

In this example, an integer variable is linked with a reference variable. When the original variable is modified, the reference also reflects the change because both refer to the same memory location.

```
int i;  
int &j = i;  
  
i = 123;  
  
cout << i;  
cout << j;
```

If i is incremented, j also changes automatically because both are the same entity in memory.

Call by Value Problem

When a function uses call by value, it receives a copy of the variables. Any modification inside the function does not affect the original variables in main.

This makes swapping or modifying original values impossible using simple parameters.

Call by Pointer vs Call by Reference

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Call by pointer solves the problem by passing addresses, allowing functions to modify original variables. However, pointer syntax is complex and less readable.

References provide a cleaner and safer alternative by directly working on original variables without explicit pointer syntax.

Call by Reference Using References

In call by reference, function parameters are declared as references:

```
void swap(int &i, int &j)
```

Inside the function, variables behave like normal integers, but they actually modify original values.

Function call:

```
swap(x, y);
```

No address operator is needed, making the code simpler and more readable.

Side Effects of References

Since references work on original data, any change inside the function affects actual variables. This can be useful (like swapping) but dangerous if unintended modifications occur.

To avoid this, const references are used when only reading data is required.

Efficiency of References

Passing large data structures by value creates copies, which is inefficient. References avoid copying and directly use original memory, improving performance.

This is especially useful for large objects like arrays or structures.

Difference Between References and Pointers

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

References must be initialized at the time of declaration, while pointers can be declared first and assigned later.

References cannot be NULL, while pointers can be NULL.

References cannot be reassigned to another variable, while pointers can change their target.

Pointer arithmetic is allowed, but reference arithmetic is not allowed.

Dangling Reference

A dangling reference occurs when a function returns a reference to a local variable that no longer exists after function execution.

This leads to undefined behavior because the reference points to invalid memory.

To avoid this, references should be returned only from global or static variables.

Example 2 – Reference Return from Function

A function returns a reference to a global variable. This allows direct modification of the variable through the function call.

```
int myNum = 0;

int& num() {
    return myNum;
}

int main() {
    num() = 200;
}
```

Here, num() acts like a variable and can be used on the left side of assignment.

Const Reference

Using const with references prevents modification of original data while still avoiding copying.

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

const bigone &r1

This provides efficiency of reference passing and safety of value passing.

Call by Value vs Reference vs Pointer

Call by value creates a copy and is safe but slow for large data.

Call by pointer passes address and allows modification but uses complex syntax.

Call by reference passes original data with clean syntax but may cause unintended changes.

Important Exam Points

References are aliases of existing variables and must be initialized at declaration.

References do not create new memory; they share the same memory location.

Call by reference using references is more efficient and cleaner than pointers.

References cannot be NULL and cannot be reassigned after initialization.

Pointer arithmetic is possible but reference arithmetic is not allowed.

Dangling references occur when returning references to local variables and must be avoided.

Const references provide safety while maintaining efficiency in function calls.

References are widely used in function parameters to implement efficient call by reference.

Lesson 30 – Reference Data Type in C++ (MCQs)

1. What is a reference in C++?

- A) A new variable created in memory
- B) An alias (another name) of an existing variable ✓**
- C) A pointer type variable
- D) A constant variable

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

2. Which of the following is TRUE about references?

- A) They can be NULL
 - B) They can be reassigned to another variable
 - C) They must be initialized at the time of declaration ✓**
 - D) They use pointer arithmetic
-

3. What happens when a reference variable is modified?

- A) Only reference value changes
 - B) Original variable remains unchanged
 - C) Original variable is also changed ✓**
 - D) A copy is created automatically
-

4. Which is a major advantage of using references in function calls?

- A) Increases memory usage
 - B) Makes program slower
 - C) Provides efficient call by reference without pointer syntax ✓**
 - D) Prevents all errors automatically
-

5. What is a dangling reference?

- A) Reference to a global variable
 - B) Reference that changes automatically
 - C) Reference pointing to a destroyed local variable ✓**
 - D) Reference to pointer variable
-

Lecture No. 31 – Operator Overloading in C++

Lecture Overview

Operator overloading allows C++ operators to be redefined for user-defined data types like classes. In this lecture, we study what operator overloading is, why it is needed, where it is used, its rules, and how it is implemented using examples.

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

What is Operator Overloading and Why is it Required

Operator overloading means giving multiple meanings to the same operator depending on the type of operands.

C++ already supports operators like +, -, *, / for built-in types such as int, float, and double. However, these operators do not work directly with objects of user-defined classes.

To solve this, we define custom behavior of operators for classes, making code more readable and natural compared to function calls like `cadd(c1, c2)`.

Where Operator Overloading is Relevant

Operator overloading is mainly used in mathematical classes such as Complex numbers, vectors, and matrices where operations like addition and subtraction are meaningful.

It is also used in non-mathematical classes like String, where + can be used for concatenation instead of functions like `strcat()`.

It improves readability and makes expressions more natural and easy to understand.

Operators to Overload

Operators in C++ are divided into unary and binary operators.

Unary operators work on a single operand such as !, ~, ++, --.

Binary operators work on two operands such as +, -, *, /, =, <, >.

C++ allows overloading of many operators including arithmetic, relational, logical, and increment/decrement operators.

Restrictions on Operator Overloading

Some operators cannot be overloaded such as ., ::, ?:, sizeof, and .*.

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

The number of operands (arity) cannot be changed.

Precedence and associativity of operators cannot be modified.

New operators cannot be created; only existing operators can be overloaded.

Operator overloading is only for user-defined types, not for primitive types.

How Operator Overloading Works

In an expression like $c1 + c2$, the left object ($c1$) calls the $+$ operator function, and the right object ($c2$) is passed as an argument.

The syntax of operator function is:

return-type operator op-symbol (parameters);

Example:

Complex operator+(Complex &c);

Member Operator Overloading Example (Complex Numbers)

In a Complex class, $+$ operator is overloaded to add real and imaginary parts.

The left object calls the function and the right object is passed as argument.

Result is stored in a temporary object and returned.

This makes expressions like:

$c3 = c1 + c2;$

simple and readable.

Assignment Operator Behavior

C++ automatically provides a default assignment operator.

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

It performs member-wise copying.

However, if a class contains dynamic memory (pointers), custom assignment operator must be defined to avoid issues.

Non-Member Operator Functions

Non-member operator functions take both operands as parameters.

If they need access to private members, they must be declared as friend functions.

They can also use public getter functions instead of direct access.

However, non-member implementations are usually slower and less efficient than member functions.

Example Program 1 – Complex Class (+ Operator)

A Complex class is created with real and imaginary parts.

The + operator is overloaded as a member function to add two complex numbers.

It returns a new Complex object containing the result.

Expression:

`c3 = c1 + c2;`

Output shows correct addition of real and imaginary parts.

Enhanced Operators (+ and +=)

The + operator can be improved by returning a reference.

The += operator directly modifies the calling object.

Example:

`c1 += c2;`

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

This is more efficient because it avoids creating temporary objects.

Example Program 2 – String Concatenation

A String class is implemented with a character array.

The + operator is overloaded to concatenate two strings using strcpy and strcat.

Expression:

string1 + string2;

produces a combined string.

This replaces traditional C-style functions like strcat() with a more readable syntax.

Key Idea of Operator Overloading

The object on the left side of the operator controls the function call.

The right-side object is passed as a parameter.

This makes operator overloading similar to function overloading in behavior but more intuitive in syntax.

Important Exam Points

Operator overloading allows user-defined types to use operators like built-in types.

It improves readability and makes code more natural and expressive.

Only existing C++ operators can be overloaded; new operators cannot be created.

Precedence, associativity, and number of operands cannot be changed.

Member operators use the left object as the calling object.

Non-member operators require both operands as arguments and are often declared as friends.

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

`+=` operator is more efficient than `+` followed by assignment.

Operator overloading is widely used in Complex numbers, Strings, and mathematical classes.

Lesson 31 – Operator Overloading (MCQs with Tick the Right One)

Q1. What is operator overloading in C++?

- A) Changing data types of variables
 - B) Giving new meaning to existing operators for user-defined types ✓
 - C) Creating new operators in C++
 - D) Removing operators from language
-

Q2. Which of the following cannot be overloaded in C++?

- A) `+`
 - B) `[]`
 - C) `::` ✓
 - D) `<<`
-

Q3. What is the main benefit of operator overloading?

- A) Increases program size
 - B) Makes code more readable and natural ✓
 - C) Replaces functions completely
 - D) Removes need for classes
-

Q4. Which statement about operator overloading is TRUE?

- A) You can change operator precedence
 - B) You can create new operators in C++
 - C) You cannot change operator associativity ✓
 - D) All operators must be global functions only
-

Q5. In a binary operator overloading function, the left operand is usually:

- A) Passed as a parameter only
- B) Ignored by compiler
- C) The object that calls the function ✓
- D) Always a primitive type

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Lecture No. 32 – Operator Overloading (Minus, Date Class, Unary Operators)

Lecture Overview

This lecture continues the concept of operator overloading in C++. We revise key ideas from previous lectures and further explore how operators can be overloaded for user-defined classes. The focus is on the minus operator, its use with classes like Complex and Date, unary operators such as increment, and comparison operators. We also emphasize when operator overloading is meaningful and how careful design is required to avoid confusing or meaningless implementations.

Recap of Operator Overloading

Operator overloading allows existing C++ operators to be used with user-defined data types. Only existing operators can be overloaded; no new operators can be created. The meaning of operators should remain consistent with their natural interpretation to keep programs readable and logical.

Binary operators work with two operands while unary operators work with one operand. The left operand in binary operators usually acts as the calling object in member functions. Overloading is similar to function overloading but applies to operators instead of function names.

Overloading Minus Operator

The minus operator is a binary operator used for subtraction. For a Complex class, subtraction means subtracting real parts and imaginary parts separately.

When overloaded as a member function, only one argument is passed because the left operand is the calling object. A temporary object is created to store the result, and the result is returned as a new Complex object.

The -= operator is different because it modifies the calling object itself. It updates the current object instead of creating a new one.

It is important to understand that overloading - does not automatically overload -=. Each operator must be defined separately.

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

In many cases, operator `--` is preferred for efficiency because it avoids creating temporary objects.

Operator Overloading with String Class

The minus operator is not naturally suitable for strings. While it can be artificially defined (such as removing a substring), such usage is often confusing and not recommended.

On the other hand, the `+` operator is very meaningful for strings because it represents concatenation. Operator overloading should always preserve natural meaning to ensure readability and maintainability.

Operators with Date Class

The Date class provides a strong real-world example of operator overloading.

Adding an integer to a date (`date + n`) means moving the date forward by `n` days. This requires handling multiple cases such as month changes, year changes, and leap years. The logic becomes complex because different months have different lengths and February depends on leap year rules.

Subtracting an integer from a date works similarly but in reverse, moving backward in time.

Subtracting two dates may represent the number of days between them, which returns an integer value instead of a Date object.

Adding two dates together generally does not make logical sense and is avoided in good design.

Careful planning is required before implementing date arithmetic because missing even one case can lead to incorrect results.

Unary Operators (Increment Operator)

Unary operators work on a single operand. In the Date class, the increment operator `++` is overloaded to move the date forward by one day.

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

In pre-increment (++date), the date is updated first and then returned. The logic checks whether the current day is the last day of the month or year and adjusts month and year accordingly.

In post-increment (date++), the original value is used first, and then the date is incremented.

A key concept used here is the this pointer, which represents the current object inside a member function.

The ++ operator can also be reused inside the + operator, showing the concept of code reuse. Instead of rewriting logic, complex operations can be built using simpler ones.

Comparison Operators with Date Class

Comparison operators such as >, <, ==, >=, and <= can also be overloaded for the Date class.

The greater than operator compares year first, then month, and finally day. If any higher-level comparison determines the result, lower-level checks are skipped.

These operators usually return a boolean value (true or false), making them easy to use in conditions like if (date1 > date2).

Comparison operators help in sorting and comparing date values in applications.

Friend Operators and Cross-Type Operations

Sometimes operators involve different types, such as integer + Date. In such cases, a member function cannot be used because the left operand is not an object of the class.

To handle this, friend functions are used. A friend operator can access private members of a class and works with parameters from both sides.

For example, integer + Date is implemented as a non-member friend function that returns a new Date object.

Friend functions are useful for combining two different classes but should be used carefully to maintain encapsulation.

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Important Exam Points

Operator overloading must preserve the original meaning of operators. Only existing operators can be overloaded and their precedence and associativity cannot be changed.

Member operators use the left operand as the calling object while non-member or friend operators take both operands as arguments.

Unary operators like ++ work on a single object and are commonly used for incrementing values such as dates.

Date arithmetic requires careful handling of months, years, and leap years, making it a complex but important real-world application.

Efficient design encourages code reuse, such as implementing + using ++ internally.

Lecture 32 – MCQs (Operator Overloading, Date Class, Unary Operators)

Q1. What is operator overloading in C++?

- A. Creating new operators in C++
- B. Changing precedence of operators
- C. Allowing existing operators to work with user-defined types ✓
- D. Removing operators from C++

Q2. Which statement about operator overloading is correct?

- A. We can create new operators like ** and %%
- B. Operator precedence can be changed
- C. Only existing operators can be overloaded ✓
- D. Operators can be used only with primitive types

Q3. In a binary member operator function, the left operand is:

- A. Passed as a parameter
- B. Ignored by the function

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

- C. The calling object ✓
 - D. Always passed by reference
-

Q4. What is the main purpose of overloading ++ in Date class?

- A. To subtract days from date
 - B. To move date forward by one day ✓
 - C. To compare two dates
 - D. To reset date to default value
-

Q5. Why is operator overloading for Date class considered complex?

- A. Because it uses pointers
 - B. Because it requires file handling
 - C. Because it must handle months, leap years, and year changes ✓
 - D. Because it cannot use functions
-

Lesson 33 – Operator Overloading, Assignment Operator, this Pointer & Conversions

Operator Overloading

Operator overloading allows us to use existing operators (like +, -, =) with user-defined classes. It helps make code simple, readable, and closer to real-world expressions. In object-oriented programming, classes are designed in such a way that they understand how to operate on their own objects. For example, writing $a + b$ for objects of a class means adding two class objects in a meaningful way defined inside the class.

Assignment Operator

C++ provides a default assignment operator that performs member-wise copying. This works fine for simple classes but creates problems when dynamic memory is involved.

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

For example, in a String class using a pointer `char *buf`, default assignment copies only the address, not actual data. This causes both objects to share the same memory, leading to errors when one object deletes the memory.

To solve this, we define our own assignment operator that:

First deletes existing memory of the calling object

Allocates new memory

Copies actual content from the source object

This ensures each object has its own independent memory.

Example (String Assignment Operator)

A String class is created with a character pointer buffer. The overloaded assignment operator ensures deep copy instead of shallow copy. It uses `new` to allocate memory and `strcpy` to copy data safely.

this Pointer

The `this` pointer is an implicit pointer available in all member functions. It points to the calling object.

It can be used in three equivalent ways:

`buf`

`this->buf`

`(*this).buf`

It helps in accessing object members and is especially useful in operator overloading.

Self Assignment

Self-assignment occurs when an object is assigned to itself like `s = s`. This can cause serious problems in memory-based classes because memory is deleted before copying, leading to invalid access.

To prevent this, we check:

`if (this == &other)`

If true, the function returns immediately without doing anything.

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Returning this Pointer

The this pointer can also be returned from functions to support chaining of operations like:

```
s3 = s2 = s1;
```

To achieve this, the assignment operator returns:

```
return *this;
```

This returns the calling object and allows multiple assignments in a single statement.

Conversions in C++

C++ allows automatic and manual type conversion between built-in types and user-defined classes.

Examples:

int to float conversion

float to double conversion

For user-defined classes, conversions can be done using:

Single-argument constructors (implicit conversion)

Conversion functions or operators (explicit conversion)

Conversion using Constructor (Fraction Class Example)

A Fraction class stores numerator and denominator separately to maintain accuracy.

A constructor with one parameter acts as a conversion constructor. For example:

```
Fraction f;
```

```
f = 3;
```

Here, integer 3 is automatically converted into Fraction (3/1).

This helps maintain precision in arithmetic operations and avoids floating-point rounding errors.

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Important Exam Points

Operator overloading improves readability and allows natural use of operators with objects.

Default assignment operator performs shallow copy, which is unsafe for dynamic memory.

Deep copy must be implemented using a custom assignment operator.

this pointer refers to the current calling object.

Self-assignment can cause memory errors and must be checked.

Returning *this enables chaining of assignments.

Constructors with one parameter act as conversion constructors.

Conversion helps in seamless interaction between built-in and user-defined types.

MCQs – Lesson 33 (Tick the Right One)

Q1. What is the main purpose of operator overloading in C++?

- A) To remove operators from programs
- B) To define new operators
- C) To make code simpler and more readable for user-defined types ✓
- D) To increase program size

Q2. What does the default assignment operator in C++ do?

- A) Deep copy of objects
- B) Member-wise (shallow) copy ✓
- C) Deletes memory automatically
- D) Converts objects into integers

Q3. Why do we need a custom assignment operator in classes with dynamic memory?

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

- A) To increase speed of program
 - B) To avoid syntax errors
 - C) To prevent shallow copy and memory issues ✓
 - D) To use fewer variables
-

Q4. What is the purpose of the `this` pointer in C++?

- A) It stores address of any variable
 - B) It points to the calling object ✓
 - C) It deletes objects automatically
 - D) It stores return value of function
-

Q5. What is the role of a single-argument constructor in a class?

- A) It destroys objects
 - B) It acts as a conversion constructor ✓
 - C) It overloads operators
 - D) It allocates arrays only
-

Lesson 34 – Arrays of Objects, Dynamic Memory, `new/delete` & Operator `[]` Overloading

Lecture Overview

In this lecture, we study how objects behave when stored in arrays. We extend the concept of arrays from primitive data types to user-defined classes. We also learn how dynamic memory allocation works for arrays of objects using `new` and `delete`.

We further explore advanced memory control by overloading `new` and `delete` operators and also learn how to overload the subscript operator `[]` to implement safe array access.

Arrays of Objects

A class is a user-defined data type, and objects are instances of classes just like variables are instances of primitive types.

An array of objects is declared in the same way as arrays of primitive types.

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Example using Date class:

```
Date myDates[10];
```

When this statement is executed:

Each element (myDates[0] to myDates[9]) is created.

Constructor is called for each object.

Since no initialization is provided, the parameterless constructor is called automatically.

If a class has no default constructor defined, the compiler generates one automatically.

Array Initialization of Objects

Objects in arrays can also be initialized at the time of declaration.

Example:

```
Date yourDate[3] = {  
    Date(10, 24, 1980),  
    Date(06, 14, 1985),  
    Date(07, 09, 1986)  
};
```

Here:

Parameterized constructor is called for each initialized object.

Remaining objects (if any) use the default constructor.

Partial Initialization Example

```
Date myDate[10] = {  
    Date(09, 03, 1970),  
    Date(08, 23, 1974)  
};
```

First two objects use parameterized constructor.

Remaining objects use parameterless constructor automatically.

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Array of Objects with String Class

```
String message[10] = {  
    "First line of message",  
    "Second line of message",  
    String("Third line of message"),  
    String()  
};
```

String literals call the constructor that accepts char*.

Uninitialized elements call the default constructor.

Dynamic Arrays of Objects

Dynamic arrays are created using new.

Example:

```
String *text;  
text = new String[5];
```

Here:

Memory is allocated for 5 objects.

Default constructor is called for each object.

Pointer text stores address of first object.

Important Limitation

We cannot pass parameters to constructors when using new[].

Only default constructor is used for all elements.

Deleting Dynamic Arrays

Correct way:

```
delete [] text;
```

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

This:

Calls destructor for all objects.

Releases full memory block.

Wrong Way (Dangerous)

delete text;

This:

Deletes only first object.

Does NOT delete full array.

Causes memory leak.

Overloading new and delete Operators

We can customize memory allocation by overloading:

new → allocation behavior

delete → deallocation behavior

Key Idea

Even if we overload:

Constructor still runs after allocation

Destructor still runs before deallocation

We can only control memory handling, not object lifecycle order

Example: Global Overloading of new/delete

```
void * operator new(size_t size)
```

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

```
{  
    return calloc(1, size);  
}
```

```
void operator delete(void *ptr)  
{  
    free(ptr);  
}
```

This version:

Initializes memory to zero

Frees memory safely

Example: Class-Level new/delete (Memory Pool)

In class Name, memory is managed using a fixed pool:

Each object gets space from pre-allocated array.

inuse[] tracks occupied slots.

new → allocates free slot

delete → frees slot

This avoids repeated heap allocation and improves performance.

Overloading Operator []

The subscript operator is used to access array elements.

Example declaration:

```
int & operator[](int index);
```

Purpose

Add safety checks for array access.

Prevent out-of-bound errors.

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Example: IntArray Class

```
int & IntArray::operator[](int index)
{
    static int dummy = 0;

    if(index >= 0 && index < length)
        return array[index];
    else
    {
        cout << "Error: index out of range";
        return dummy;
    }
}
```

Important Point

Returning a reference allows:

```
numbers[i] = 10;
```

So numbers[i] can be used on left-hand side.

Important Exam Points

Arrays of objects call constructors for each element.

Dynamic arrays use new[] and must be deleted with delete[].

Wrong delete usage causes memory leaks.

new returns void* and takes size_t.

delete frees memory and calls destructors.

Operator overloading allows custom memory management.

Operator [] can be overloaded for boundary checking.

Returning reference enables assignment to array elements.

5 MCQs (Important)

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Q1. What happens when an array of objects is created using `new`?

- A) Only memory is allocated
 - B) Only constructor of first object is called
 - C) Constructor is called for each object ✓
 - D) Destructor is called immediately
-

Q2. Which operator is correct for deleting an array of objects?

- A) `delete`
 - B) `delete[]` ✓
 - C) `free[]`
 - D) `remove[]`
-

Q3. What is returned by the `new` operator?

- A) `int`
 - B) `object`
 - C) `void*` ✓
 - D) `reference`
-

Q4. What happens if `delete` is used instead of `delete[]`?

- A) All objects are deleted properly
 - B) Only first object is deleted ✓
 - C) Compiler error always occurs
 - D) Memory is doubled
-

Q5. Why is operator `[]` overloaded in a class?

- A) To slow program
 - B) To access private data only
 - C) To perform boundary checking ✓
 - D) To delete arrays
-

Lesson 34 – Arrays of Objects, Dynamic Memory, `new/delete` & Operator `[]` Overloading

Lecture Overview

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

In this lecture, we study how objects behave when stored in arrays. We extend the concept of arrays from primitive data types to user-defined classes. We also learn how dynamic memory allocation works for arrays of objects using `new` and `delete`.

We further explore advanced memory control by overloading `new` and `delete` operators and also learn how to overload the subscript operator `[]` to implement safe array access.

Arrays of Objects

A class is a user-defined data type, and objects are instances of classes just like variables are instances of primitive types.

An array of objects is declared in the same way as arrays of primitive types.

Example using `Date` class:

```
Date myDates[10];
```

When this statement is executed:

Each element (`myDates[0]` to `myDates[9]`) is created.

Constructor is called for each object.

Since no initialization is provided, the parameterless constructor is called automatically.

If a class has no default constructor defined, the compiler generates one automatically.

Array Initialization of Objects

Objects in arrays can also be initialized at the time of declaration.

Example:

```
Date yourDate[3] = {  
    Date(10, 24, 1980),  
    Date(06, 14, 1985),  
    Date(07, 09, 1986)  
};
```

Here:

Parameterized constructor is called for each initialized object.

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Remaining objects (if any) use the default constructor.

Partial Initialization Example

```
Date myDate[10] = {  
    Date(09, 03, 1970),  
    Date(08, 23, 1974)  
};
```

First two objects use parameterized constructor.

Remaining objects use parameterless constructor automatically.

Array of Objects with String Class

```
String message[10] = {  
    "First line of message",  
    "Second line of message",  
    String("Third line of message"),  
    String()  
};
```

String literals call the constructor that accepts char*.

Uninitialized elements call the default constructor.

Dynamic Arrays of Objects

Dynamic arrays are created using `new`.

Example:

```
String *text;  
text = new String[5];
```

Here:

Memory is allocated for 5 objects.

Default constructor is called for each object.

Pointer `text` stores address of first object.

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Important Limitation

We cannot pass parameters to constructors when using `new[]`.

Only default constructor is used for all elements.

Deleting Dynamic Arrays

Correct way:

`delete [] text;`

This:

Calls destructor for all objects.

Releases full memory block.

Wrong Way (Dangerous)

`delete text;`

This:

Deletes only first object.

Does NOT delete full array.

Causes memory leak.

Overloading new and delete Operators

We can customize memory allocation by overloading:

`new` → allocation behavior

`delete` → deallocation behavior

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Key Idea

Even if we overload:

Constructor still runs after allocation

Destructor still runs before deallocation

We can only control memory handling, not object lifecycle order

Example: Global Overloading of new/delete

```
void * operator new(size_t size)
{
    return calloc(1, size);
}
```

```
void operator delete(void *ptr)
{
    free(ptr);
}
```

This version:

Initializes memory to zero

Frees memory safely

Example: Class-Level new/delete (Memory Pool)

In class Name, memory is managed using a fixed pool:

Each object gets space from pre-allocated array.

inuse[] tracks occupied slots.

new → allocates free slot

delete → frees slot

This avoids repeated heap allocation and improves performance.

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Overloading Operator []

The subscript operator is used to access array elements.

Example declaration:

```
int & operator[](int index);
```

Purpose

Add safety checks for array access.

Prevent out-of-bound errors.

Example: IntArray Class

```
int & IntArray::operator[](int index)
{
    static int dummy = 0;

    if(index >= 0 && index < length)
        return array[index];
    else
    {
        cout << "Error: index out of range";
        return dummy;
    }
}
```

Important Point

Returning a reference allows:

```
numbers[i] = 10;
```

So numbers[i] can be used on left-hand side.

Important Exam Points

Arrays of objects call constructors for each element.

Dynamic arrays use new[] and must be deleted with delete[].

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Wrong delete usage causes memory leaks.

new returns void* and takes size_t.

delete frees memory and calls destructors.

Operator overloading allows custom memory management.

Operator [] can be overloaded for boundary checking.

Returning reference enables assignment to array elements.

5 MCQs (Important)

Q1. What happens when an array of objects is created using new?

- A) Only memory is allocated
- B) Only constructor of first object is called
- C) Constructor is called for each object ✓
- D) Destructor is called immediately

Q2. Which operator is correct for deleting an array of objects?

- A) delete
- B) delete[] ✓
- C) free[]
- D) remove[]

Q3. What is returned by the new operator?

- A) int
- B) object
- C) void* ✓
- D) reference

Q4. What happens if delete is used instead of delete[]?

- A) All objects are deleted properly
 - B) Only first object is deleted ✓
 - C) Compiler error always occurs
 - D) Memory is doubled
-

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Q5. Why is operator [] overloaded in a class?

- A) To slow program
- B) To access private data only
- C) To perform boundary checking ✓
- D) To delete arrays

Lesson 35 – Streams in C++ (Input/Output, Formatted I/O, Buffered I/O & Stream Functions)

Lecture Overview

In this lecture, we study the concept of **streams in C++**, which form the core of input and output operations.

We explore how data flows between the program and external devices using streams like cin, cout, cerr, and clog.

We also study formatted I/O, stream states, buffering concepts, and important stream member functions such as get(), getline(), read(), and write().

Streams

A stream is an **ordered sequence of bytes** used to transfer data between a program and external devices.

Streams are used for:

- Keyboard input
- Screen output
- File input/output
- Memory communication

In C++, streams are implemented as **objects**, making I/O operations object-oriented and flexible.

Types of Streams

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

There are two main types:

Input Streams → used to read data into the program (cin)

Output Streams → used to display or store data (cout)

Basic Concept of Stream

A stream works like a **pipeline or queue (FIFO)**.

Data enters first → exits first

Example:

Keyboard input is converted into ASCII code and then stored in variables.

So streams handle **conversion + transfer + storage** automatically.

Source and Destination of Streams

Every stream has:

Source → where data comes from

Destination → where data goes

cin

Source: Keyboard

Destination: Program variables

cout

Source: Program variables

Destination: Screen

So streams always connect **input source and output destination**.

Stream State

Streams maintain an internal state to detect errors.

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

If invalid input is given (e.g., character instead of integer), the stream enters a **fail state**.

Programs can check stream state to ensure correct execution and prevent errors.

Example:

Division by zero or invalid input can be detected using stream state checking.

Formatted Input and Output

Streams support **formatted I/O**, which controls how data is displayed.

Formatting includes:

- Controlling decimal places
- Aligning output
- Controlling width and spacing
- Structured display (tables, bills, reports)

Example:

Instead of printing full precision, we can display:

3.14159 → 3.141

Formatted output improves readability and presentation.

Stream Operators (<< and >>)

C++ uses overloaded operators for input/output:

- extraction operator (input)
- << → insertion operator (output)

Example:

```
cin >> i;  
cout << i;
```

These operators are overloaded for different data types like int, char, float, and string.

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Operator Chaining in Streams

Streams support chaining:

```
cout << "Value is " << x;  
cin >> x >> y;
```

This works because:

Each operator returns a reference to the same stream object (cin or cout).

This allows multiple operations in a single statement.

Buffered and Unbuffered I/O

Buffered Output

Data is stored temporarily and written in bulk for efficiency.

Unbuffered Output

Data is displayed immediately.

cout

Buffered output stream

cerr

Unbuffered output stream (used for errors and debugging)

endl vs \n

\n → moves cursor to next line

endl → moves to next line + flushes buffer

So endl forces immediate output display.

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Standard Stream Objects

cin → Standard input (keyboard)

cout → Standard output (screen)

cerr → Unbuffered error output

clog → Buffered error output

All are predefined stream objects in C++.

Stream Member Functions (cin)

get()

Reads a single character.

getline()

Reads a full line until delimiter (usually newline).

cin.getline(buffer, size);

read()

Reads a block of characters.

peek()

Checks next character without removing it.

unget()

Returns last extracted character back to stream.

Stream Member Functions (cout)

put()

Outputs a single character.

write()

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Outputs a block of data without formatting.

getline Function

Used to read full sentences including spaces.

Example:

```
cin.getline(buffer, SIZE);
```

It reads until:

Buffer limit is reached OR newline character is found

Unlike `cin >>`, it does not stop at spaces.

Important Behavior of cin

cin treats **whitespace (space, tab, newline)** as delimiter.

Example:

Input: naveed malik

Only “naveed” is stored using `cin >> name`.

Remaining text stays in buffer.

To read full names, `getline()` must be used.

Example of Buffered Input (read/write)

```
cin.read(buffer, 20);  
cout.write(buffer, cin.gcount());
```

`read()` → reads fixed number of characters

`write()` → prints raw data

`gcount()` → returns number of characters read

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Recap of Streams

Streams are:

- Ordered byte sequences
- Used for input/output
- Connected between program and devices
- Implemented as C++ objects
- Support operator overloading
- Support formatting and buffering

Important Exam Points

Streams are ordered sequence of bytes used for I/O.

cin is input stream and cout is output stream.

Streams support operator overloading (<< and >>).

endl flushes output buffer.

cerr is unbuffered output stream.

Streams support formatted and unformatted I/O.

getline is used to read full line input.

cin treats space as delimiter.

Stream operators return reference for chaining.

Buffered output improves performance.

5 MCQs (Important)

Q1. What is a stream in C++?

- A) A variable type
 - B) A function
 - C) Ordered sequence of bytes ✓
 - D) A loop structure
-

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Q2. Which stream is unbuffered?

- A) cin
 - B) cout
 - C) cerr ✓
 - D) clog
-

Q3. What does endl do?

- A) Only prints new line
 - B) Flushes buffer and prints new line ✓
 - C) Deletes output
 - D) Stops program
-

Q4. Which operator is used for output stream?

- A) >>
 - B) << ✓
 - C) ::
 - D) ->
-

Q5. What does getline() do?

- A) Reads one word
 - B) Reads full line including spaces ✓
 - C) Deletes input
 - D) Writes output
-

Lesson 36 – Stream Manipulations, Manipulators & Formatting in C++

Lecture Overview

In this lecture, we study how C++ streams can be manipulated to control input and output formatting. We explore stream states, manipulators, format flags, and different techniques used to control how data is displayed or read.

We also learn how numbers can be represented in different bases, how output can be aligned, and how scientific formatting is applied in real programming scenarios.

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Stream Manipulations

Stream manipulation refers to controlling the behavior of input and output streams like `cin` and `cout`.

To use stream manipulators, we include:

```
#include <iomanip.h>
```

Streams are sequences of bytes that transfer data between source and destination while also performing formatting and conversion.

Streams also maintain internal state flags that indicate whether operations succeed or fail.

Stream State Checking

C++ stream objects maintain internal status flags that describe their state.

End of File Check

```
cin.eof();
```

Returns true when input reaches end of file.

Fail State

```
cin.fail();
```

Indicates whether an input operation failed due to formatting or invalid input.

Bad State

```
cin.bad();
```

Indicates serious data loss or stream corruption.

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Good State

```
cin.good();
```

Returns true when stream is in a valid working state.

Clearing Stream Errors

```
cin.clear();
```

Resets all error flags and restores stream to normal state.

Manipulators

Manipulators are special instructions inserted into streams that change formatting behavior.

They are used with << or >> and affect how input/output is processed.

Non-Parameterized Manipulators

These manipulators do not take arguments.

Number Base Manipulators

```
cout << dec;  
cout << hex;  
cout << oct;
```

They convert number representation:

dec → Decimal (base 10)
hex → Hexadecimal (base 16)
oct → Octal (base 8)

Example:

```
int i = 10;  
cout << oct << i; // outputs 12  
cout << hex << i; // outputs A
```

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

endl Manipulator

```
cout << endl;
```

Inserts new line and flushes output buffer.

flush Manipulator

```
cout << flush;
```

Forces immediate output of buffered data.

ends Manipulator

```
cout << ends;
```

Adds null character to output stream.

ws Manipulator

```
cin >> ws;
```

Skips leading whitespace characters.

Why Base Manipulation is Important

Different number systems are used in computing:

Binary (base 2)

Octal (base 8)

Decimal (base 10)

Hexadecimal (base 16)

These systems align closely with how computers store data in bits and bytes.

Parameterized Manipulators

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

These manipulators take arguments to control formatting.

setw (Set Width)

```
cout << setw(4) << number;
```

Sets output width and right aligns by default.

setfill (Fill Character)

```
cout << setfill('*') << setw(10) << amount;
```

Fills empty spaces with a specified character.

Example:

```
*****4000
```

setprecision

```
cout << setprecision(2) << pi;
```

Controls number of digits shown for floating-point values.

Example:

3.1415926 becomes 3.14

setbase

```
cout << setbase(8);  
cout << setbase(16);  
cout << setbase(10);
```

Alternative to oct, hex, dec manipulators.

Format State Flags

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Streams use internal flags to control formatting behavior.

These are controlled using:

```
cout.setf();  
cout.unsetf();
```

Common Format Flags

ios::left → Left justify output
ios::right → Right justify output
ios::internal → Padding between sign and number
ios::showbase → Shows base prefix (0, 0x)
ios::showpoint → Forces decimal point display
ios::uppercase → Uses uppercase letters
ios::scientific → Scientific notation
ios::fixed → Fixed decimal format

Text Alignment Example

```
cout.setf(ios::left, ios::adjustfield);  
cout << setw(10) << i;
```

Output can be left, right, or internal aligned.

Base Display with showbase

```
cout.setf(ios::showbase);  
cout.setf(ios::oct, ios::basefield);
```

Output example:

```
0115  
0x4d  
77
```

This clearly shows number base representation.

Scientific Notation

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

For very large numbers:

```
cout.setf(ios::scientific, ios::floatfield);
```

Example:

```
1.946000e+009
```

To revert:

```
cout.setf(ios::fixed, ios::floatfield);
```

Uppercase Control

```
cout.setf(ios::uppercase);
```

Converts e to E in scientific notation and hex letters to uppercase.

Member Functions vs Manipulators

setw(n) → width(n)

setprecision(n) → precision(n)

setfill(c) → fill(c)

Example:

```
cout.width(10);  
cout.fill('*');  
cout.precision(2);
```

Important Exam Points

Manipulators control formatting of input/output streams.

Streams maintain state using flags like fail, good, eof, bad.

Non-parameterized manipulators include dec, hex, oct, endl, flush.

Parameterized manipulators include setw, setfill, setprecision, setbase.

showbase displays number base prefixes like 0x and 0.

scientific format is used for very large or small numbers.

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

setf() function is used to control formatting flags.

Member functions provide alternative to manipulators.

5 MCQs (Important)

Q1. Which header file is required for manipulators like setw and setprecision?

- A) iostream.h
- B) fstream.h
- C) iomanip.h ✓
- D) stdlib.h

Q2. What does endl do?

- A) Only new line
- B) Only flush buffer
- C) New line + flush buffer ✓
- D) Clears stream

Q3. What is the purpose of setw()?

- A) Sets precision
- B) Sets output width ✓
- C) Sets base
- D) Sets buffer size

Q4. Which manipulator shows hexadecimal values?

- A) dec
- B) oct
- C) hex ✓
- D) setbase(2)

Q5. What does ios::scientific do?

- A) Fixed point output
 - B) Integer formatting
 - C) Scientific notation output ✓
 - D) Binary conversion
-

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Lecture No. 37 – Stream Manipulations, Manipulators & Formatting in C++

Lecture Overview

In this lecture, we study how C++ streams can be manipulated to control input and output formatting. We explore stream states, manipulators, format flags, and different techniques used to control how data is displayed or read.

We also learn how numbers can be represented in different bases, how output can be aligned, and how scientific formatting is applied in real programming scenarios.

Overloading Insertion and Extraction Operators

We are already aware that while overloading operators, functions are written and spirit or behavior of the operators (+, -, *, /) is maintained in their implementations. Similarly the operator's spirit is kept intact while overloading stream insertion and extraction operators.

We get an integer as input by writing the following lines:

```
int i;  
cin >> i;
```

Have a look on the stream extraction operator's (>>) behavior here. The similar behavior is maintained when we overload this stream extraction operator (>>) or stream insertion operator (<<).

There are couple of important things to take care of, before starting implementation for overloading an operator:

The first thing to see is the type of the operator i.e. whether the operator is binary or unary. The binary operator takes two operands while unary operator takes one. The number of operands for an operator cannot be changed while overloading it.

Secondly, the programmer has to take care of, what an operator is returning back. For example, in case of addition (+), it returns back the result of addition. So the cascading statement like `a + b + c;` can be executed successfully. In this case, at first, `b + c` is executed. The result of this operation is returned by the operator +, to add it in the variable `a`. So in actual, the operation is carried out as:

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

$a + (b + c)$

We want to overload stream extraction (>>) and insertion (<<) operators which are actually already overloaded.

You can see the lines:

```
int i = 123;  
double d = 456.12;  
float f = 789.1;
```

```
cout << i << " ";  
cout << d << " ";  
cout << f;
```

Alternatively:

```
cout << i << " " << d << " " << f;
```

Similarly:

```
cin >> i >> d >> f;
```

The file `iostream.h` contains the operator overloading declarations for these stream insertion (<<) and extraction (>>) operators for native data types.

In order to use these operators with classes, we have to overload them.

There are two ways of overloading operators, either as class members or non-members. But these insertion (<<) and extraction (>>) operators cannot be overloaded as members because the left side operand is `cin` or `cout`.

Therefore, these must be overloaded as non-members.

We can either use getters/setters or declare them as friend functions.

Normally friend functions are defined outside the class after declaration.

The object on the left of operator will be stream object and right will be class object.

The return type must support cascading operations.

For insertion operator:

```
ostream & operator << (ostream & output, Vehicle v);
```

The first parameter is passed by reference because streams cannot be copied.

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

We return ostream by reference to support cascading.

Inside friend function we can access private members:

```
output << v.tyre;
```

This works because tyre is private but accessible to friend.

Example 1 – Date Class

```
#include <iostream.h>
class Date
{
    friend ostream & operator << (ostream & os, Date d);
    friend istream & operator >> (istream & is, Date & d);

private:
    int month, day, year;
};

ostream & operator << (ostream & os, Date d)
{
    os << d.day << "." << d.month << "." << d.year;
    return os;
}

istream & operator >> (istream & is, Date & d)
{
    cout << "\n Enter day: ";
    cin >> d.day;

    cout << " Enter month: ";
    cin >> d.month;

    cout << " Enter year: ";
    cin >> d.year;

    return is;
}

int main()
{
    Date date1, date2;

    cin >> date1 >> date2;
    cout << date1 << date2;
}
```

Output:

Parameterless constructor called

Enter two dates

Enter day of the date: 14

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Enter month: 12
Enter year: 1970
Enter day of the date: 05
Enter month: 09
Enter year: 2000
Entered date1 is: 14.12.1970
Entered date2 is: 5.9.2000

Example 2 – Matrix Class

Without operator overloading:

Matrix input/output is done using functions:

```
getMatrix()  
displayMatrix()
```

This is not consistent with built-in types like:

```
cin >> i  
cout << i
```

With operator overloading:

```
class Matrix  
{  
    float elements[30][30];  
    int numRows, numCols;  
  
public:  
    Matrix(int rows = 0, int cols = 0)  
    {  
        numRows = rows;  
        numCols = cols;  
    }  
  
    friend ostream & operator << (ostream &, Matrix &);  
    friend istream & operator >> (istream &, Matrix &);  
};  
istream & operator >> (istream & input, Matrix & m)  
{  
    for(int i = 0; i < m.numRows; i++)  
        for(int j = 0; j < m.numCols; j++)  
            input >> m.elements[i][j];  
  
    return input;  
}
```

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

```
}
ostream & operator << (ostream & output, Matrix & m)
{
    for(int r = 0; r < m.numRows; r++)
    {
        for(int c = 0; c < m.numCols; c++)
            output << m.elements[r][c] << "\t";

        output << endl;
    }

    return output;
}

int main()
{
    Matrix matrix(3,3);

    cin >> matrix;
    cout << matrix;
}
```

Tips

Stream insertion and extraction operators allow objects to behave like built-in types.

These operators must be overloaded as non-member functions.

Friend functions are used to access private data.

Cascading works because operators return stream references.

<< returns ostream & and >> returns istream &.

Important Exam Points

Insertion operator (<<) is used for output.

Extraction operator (>>) is used for input.

Operators must be overloaded as non-member functions.

Friend keyword allows access to private data.

Cascading is possible due to return by reference.

Return type must be ostream & or istream &.

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

5 MCQs (Important)

Q1. Why cannot << be overloaded as a member function?

- A) It is unary
- B) Left operand is cout object ✓
- C) It is private
- D) It is static

Q2. What does operator << return?

- A) int
- B) object
- C) ostream & ✓
- D) void

Q3. Why are friend functions used?

- A) For speed
- B) To access private members ✓
- C) To overload variables
- D) To delete objects

Q4. What enables cascading in stream operators?

- A) pointers
- B) recursion
- C) return by reference ✓
- D) loops

Q5. Which operator is used for input?

- A) <<
- B) >> ✓
- C) ++
- D) --

**Lecture No. 38 – User Defined Manipulators & Static
Keyword in C++**

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Lecture Overview

In this lecture, we study user-defined manipulators and the concept of static keyword in C++. We explore how manipulators can be created by programmers to control output formatting and how static variables, objects, and data members help in maintaining state in programs.

We also learn how static behaves differently in functions, objects, and classes and how it is useful in real programming situations.

User Defined Manipulators

We have already studied built-in manipulators like endl, setw, hex, and dec. User defined manipulators are programmer-defined functions that behave like manipulators and are used with stream insertion operator <<.

These manipulators are used to format output such as inserting tabs, spaces, new lines, symbols, or applying custom formatting.

A user defined manipulator works with ostream objects like cout and must return ostream reference to support cascading.

General format:

```
ostream& manipulator_name(ostream& os)
```

The stream object is passed as argument and returned after modification so chaining continues smoothly.

Examples of User Defined Manipulator

We can create manipulators for different tasks like sound, spacing, and formatting.

Example:

```
ostream & bell(ostream & output)
{
    return output << '\a';
}
```

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

```
ostream & tab(ostream & output)
{
    return output << '\t';
}
```

```
ostream & endLine(ostream & output)
{
    return output << '\n' << flush;
}
```

Usage:

```
cout << "Virtual " << tab << "University" << bell << endLine;
```

Each manipulator performs a small formatting task and returns the same stream.

Matrix Example Using User Defined Manipulators

```
#include <iostream.h>
#include <iomanip.h>

class Matrix
{
private:
    int numRows;
    int numCols;
    float elements[3][3];

public:
    Matrix(int rows = 0, int cols = 0)
    {
        numRows = rows;
        numCols = cols;
    }

    friend ostream & operator << (ostream &, Matrix &);
    friend istream & operator >> (istream &, Matrix &);

    friend ostream & spaceFirst(ostream &);
    friend ostream & spaceBetween(ostream &);
    friend ostream & line(ostream &);
    friend ostream & newLine(ostream &);
    friend ostream & star(ostream &);
    friend ostream & sound(ostream &);
};

istream & operator >> (istream & input, Matrix & m)
{
    for(int i = 0; i < m.numRows; i++)
        for(int j = 0; j < m.numCols; j++)
            input >> m.elements[i][j];
}
```

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

```
    return input;
}
ostream & operator << (ostream & output, Matrix & m)
{
    for(int i = 0; i < 60; i++)
    {
        if(i == 30)
            output << "Displaying The Matrix";
        else
            output << star;
    }

    output << newLine;

    for(int r = 0; r < m.numRows; r++)
    {
        output << spaceFirst << line;

        for(int c = 0; c < m.numCols; c++)
        {
            output << spaceBetween << m.elements[r][c] << sound << spaceBetween;
        }

        output << spaceBetween << line;
        output << newLine;
    }

    return output;
}
ostream & spaceFirst(ostream & output)
{
    output << setw(33);
    return output;
}

ostream & spaceBetween(ostream & output)
{
    output << setw(4);
    return output;
}

ostream & line(ostream & output)
{
    return output << "|";
}

ostream & newLine(ostream & output)
{
    return output << endl;
}

ostream & star(ostream & output)
{
    return output << "*";
}
```

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

```
ostream & sound(ostream & output)
{
    return output << "\a";
}
```

Static Keyword

The static keyword is used for variables that maintain their value throughout program execution.

A static variable inside a function is created only once and retains its value between function calls.

Example:

```
static int i = 0;
```

It is initialized only once and retains updates between calls.

Unlike automatic variables, static variables are not destroyed when function ends.

They are stored in static memory area and exist until program terminates.

Static Variable Example

```
#include <iostream.h>

void staticVarFun();
void nonstaticVarFun();

void main()
{
    for(int i = 0; i < 10; i++)
        staticVarFun();

    for(int j = 0; j < 10; j++)
        nonstaticVarFun();
}

void staticVarFun()
{
    static int i = 0;
    i++;
    cout << "The value of i is: " << i << endl;
}

void nonstaticVarFun()
{

```

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

```
int i = 0;  
i++;  
cout << "The value of i is: " << i << endl;  
}
```

Output

The value of i is: 1
The value of i is: 2
The value of i is: 3
The value of i is: 4
The value of i is: 5
The value of i is: 6
The value of i is: 7
The value of i is: 8
The value of i is: 9
The value of i is: 10

The value of i is: 1
The value of i is: 1
The value of i is: 1
The value of i is: 1
The value of i is: 1
The value of i is: 1
The value of i is: 1
The value of i is: 1
The value of i is: 1
The value of i is: 1

Static Objects

Static objects are objects created using static keyword inside a function.

They are constructed only once and destroyed only when program ends.

Unlike normal objects, they retain their state across function calls.

Example:

```
class truck {  
private:  
    char name;  
  
public:  
    truck(char c) : name(c)
```

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

```
{
    cout << "inside constructor of " << name << endl;
}

~truck()
{
    cout << "inside destructor of " << name << endl;
}
};

truck A('A');

void f()
{
    truck C('C');
}

void g()
{
    static truck D('D');
}

int main()
{
    truck B('B');

    f();
    g();
}
```

Static Data Members of a Class

Static data members belong to class not objects.

Only one copy exists for all objects.

They are initialized outside class using scope resolution operator:

className::variable = value;

Example:

savingsAccount::profit_rate = 3.0;

If one object changes it, all objects are affected.

Static members are shared across all objects of class.

Example use: counting objects.

class student

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

```
{  
public:  
    static int how_many;  
  
    student()  
    {  
        how_many++;  
    }  
  
    ~student()  
    {  
        how_many--;  
    }  
};
```

Important Exam Points

User defined manipulators return ostream& and are used for formatting output.

Static variables retain values between function calls.

Static objects are created once and destroyed at program end.

Static data members belong to class and are shared among all objects.

Static data members are initialized outside class using scope resolution operator.

MCQs

Q1. What is returned by a user defined manipulator?

- A) int
 - B) void
 - C) ostream & ✓
 - D) class object
-

Q2. What does static variable inside function do?

- A) resets every call
 - B) keeps previous value ✓
 - C) deletes memory
 - D) becomes global automatically
-

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Q3. Static data member belongs to:

- A) object
 - B) function
 - C) class ✓
 - D) pointer
-

Q4. Static object is destroyed when:

- A) function ends
 - B) loop ends
 - C) program ends ✓
 - D) class ends
-

Q5. Why static variables are stored separately?

- A) stack memory
 - B) heap memory
 - C) static memory area ✓
 - D) register memory
-

Lecture No. 39 – Pointers, References, Dynamic Memory Allocation, Copy Constructor (Revision Lecture)

Lecture Overview

In this lecture, we revise important concepts including pointers, references, call by value, call by reference, dynamic memory allocation, copy constructor, and assignment behavior in C++.

These topics are essential for understanding memory management and object behavior in C++ programs. The revision helps strengthen core programming concepts before moving to advanced topics.

Pointers

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Pointer is a special type of variable that stores a memory address instead of a value.

A pointer does not contain actual data; it contains the address of another variable.

In C++, pointers can be of different types such as int pointer, float pointer, char pointer, or even class pointer.

A pointer to an integer stores the address of an integer variable. Similarly, arrays can also be accessed using pointers by pointing to the first element.

The address-of operator (&) is used to get the memory address of a variable.

Pointers are widely used for dynamic memory allocation and efficient memory handling.

References

A reference is an alias of an existing variable.

It directly refers to the original variable and must always be initialized.

Unlike pointers, references cannot be null.

References are safer than pointers and are commonly used in call by reference.

The & symbol is used to declare a reference in function parameters.

Call by Value

In call by value, a copy of the actual parameter is passed to the function.

Any changes made inside the function do not affect the original variable.

This method is safe but less efficient for large objects because it creates copies.

Call by Reference

In call by reference, the actual memory address is passed to the function.

Any changes made inside the function affect the original variable.

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

It is more efficient because no copy is created.

It is used when we want to modify original data or avoid copying large objects.

Const reference can be used when we want efficiency without modifying data.

Dynamic Memory Allocation

Dynamic memory allocation allows memory to be allocated at runtime from the heap (free store).

C++ provides two operators for this purpose: new and delete.

The new operator allocates memory and returns a pointer.

The delete operator frees the allocated memory.

For arrays, new [] is used for allocation and delete [] is used for deallocation.

If memory is not freed properly, it causes memory leaks.

Example of Dynamic Memory Allocation

```
int *p;  
p = new int;
```

```
*p = 10;  
cout << *p;
```

```
delete p;
```

For arrays:

```
int *p;  
p = new int[10];
```

```
p[0] = 5;
```

```
delete [] p;
```

Dynamic Memory in Classes

When using dynamic memory inside a class, we use pointers as data members.

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Memory is allocated in constructor using new.

Memory must be freed in destructor using delete [].

Example structure:

```
class Matrix
{
    int *m;
    int row, col;

public:
    Matrix(int rows, int cols)
    {
        m = new int[rows * cols];
    }

    ~Matrix()
    {
        delete [] m;
    }
};
```

Assignment Problem (Shallow Copy Issue)

If we assign one object to another using default assignment, only pointer values are copied.

Both objects then point to the same memory.

If one object deletes memory, the other becomes invalid.

This causes serious runtime errors.

Therefore, we must overload assignment operator for deep copy.

Copy Constructor

A copy constructor creates a new object using an existing object.

It performs deep copy instead of shallow copy.

It ensures each object has its own memory.

Syntax:

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

ClassName(const ClassName &obj);

Copy Constructor Example (String Class)

```
#include <iostream.h>
#include <string.h>
#include <stdlib.h>

class String
{
    char *c;

public:

    String(const char *s)
    {
        c = new char[30];
        strcpy(c, s);
    }

    String(const String &other)
    {
        int len = strlen(other.c);
        c = new char[len + 1];
        strcpy(c, other.c);
    }

    void print()
    {
        cout << c << endl;
    }

    void copy(char *s)
    {
        strcpy(c, s);
    }

    ~String()
    {
        delete [] c;
    }
};

int main()
{
    String s1("test1");
    String s2(s1);

    cout << "s1: ";
    s1.print();

    cout << "s2: ";
    s2.print();
}
```

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

```
s1.copy("new string");  
  
cout << "After change:" << endl;  
  
cout << "s1: ";  
s1.print();  
  
cout << "s2: ";  
s2.print();  
}
```

Assignment Operator Issue

If assignment operator is not overloaded, shallow copy occurs.

This leads to shared memory problems.

Self-assignment must also be handled carefully.

Example:

```
m1 = m2;
```

If `m1 = m1` happens, it can cause deletion issues if not handled properly.

Rules for Dynamic Memory Allocation in Classes

A class using dynamic memory must always have constructor, destructor, and copy constructor.

Assignment operator must be overloaded for deep copy.

Destructor must free allocated memory using `delete []`.

Usage of Copy Constructor

Copy constructor is used when:

An object is created from another object.

An object is passed by value to a function.

An object is returned from a function.

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Example:

Matrix m2 = m1;

This calls copy constructor, not assignment operator.

Important Exam Points

Pointers store memory addresses.

References are aliases of variables.

Call by value creates copies; call by reference modifies original data.

new allocates memory; delete frees memory.

Dynamic memory is allocated at runtime.

Classes with pointers must define destructor, copy constructor, and assignment operator.

Copy constructor performs deep copy.

Shallow copy leads to memory errors.

MCQs

Q1. What does a pointer store?

- A) Value
 - B) Address ✓
 - C) Function
 - D) Class
-

Q2. Which operator allocates memory in C++?

- A) malloc
 - B) delete
 - C) new ✓
 - D) free
-

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Q3. What does delete do?

- A) Allocates memory
 - B) Frees memory ✓
 - C) Creates object
 - D) Copies object
-

Q4. Copy constructor is used to:

- A) Delete object
 - B) Create deep copy ✓
 - C) Free memory
 - D) Assign values only
-

Q5. Problem with shallow copy is:

- A) Fast execution
 - B) Shared memory ✓
 - C) No objects
 - D) More memory
-

Lecture No. 40 – Objects as Class Members, Structures, and Nested Classes

Lecture Overview

In this lecture, we study how objects of one class can be used inside another class as data members. We also explore construction and destruction order of objects, initializer lists, structures as class members, and nested (inner) classes. These concepts help in writing modular, reusable, and well-structured C++ programs.

Objects as Class Members

A class is a user-defined data type and it can be used inside other classes in the same way as built-in data types. This means that objects of one class can become data members of another class.

When a class contains objects of other classes, construction and destruction order becomes very important. First, the contained objects are constructed, and then the

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

outer class object is constructed. At destruction time, the outer object is destroyed first and then the contained objects are destroyed.

Construction and Destruction Order Example

```
#include <iostream.h>

class A
{
public:
    A()
    {
        cout << "A Constructor\n";
    }

    ~A()
    {
        cout << "A Destructor\n";
    }
};

class B
{
    A a;
public:
    B()
    {
        cout << "B Constructor\n";
    }

    ~B()
    {
        cout << "B Destructor\n";
    }
};

void main()
{
    B b;
}
```

Output:

```
A Constructor
B Constructor
B Destructor
A Destructor
```

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Initializer List

Initializer list is used to initialize member objects at the time of construction. It is placed after the constructor header using a colon (:).

It is more efficient than assigning values inside the constructor body because member objects are initialized directly.

Example – PersonInfo Class

```
class Date
{
public:
    Date(int m, int d, int y);
};

class PersonInfo
{
    char name[30];
    char address[60];
    Date birthday;

public:
    PersonInfo(char *nm, char *addr, int m, int d, int y)
        : birthday(m, d, y)
    {
        strcpy(name, nm);
        strcpy(address, addr);
    }
};
```

In this example, the birthday object is initialized using the initializer list.

Order in Initializer List

Even if initializer list is written in any order, objects are initialized in the order they are declared inside the class.

Example – Column, Row, Matrix

```
class Column
{
public:
    Column(int s)
```

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

```
{  
    cout << "Column created\n";  
}  
};
```

```
class Row  
{  
    Column col;  
  
public:  
    Row(int r, int c) : col(c)  
    {  
        cout << "Row created\n";  
    }  
};
```

```
class Matrix  
{  
    Row row;  
  
public:  
    Matrix(int r, int c) : row(r, c)  
    {  
        cout << "Matrix created\n";  
    }  
};
```

Output shows construction order:
Column → Row → Matrix
Destruction order is reverse:
Matrix → Row → Column

Advantages of Objects as Class Members

Code reusability is improved because existing classes can be reused inside new classes.

Code becomes modular and easier to manage.

Complex systems can be broken into smaller components.

Testing becomes easier because individual classes are tested separately.

Structures as Class Members

Structures can also be used inside classes just like objects.

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

The only difference between class and structure in C++ is default access level.

Structures have public access by default, while classes have private access by default.

Example – Structure Inside Class

```
struct VehicleParts
{
    int wheels;
    int seats;

    VehicleParts(int w, int s)
    {
        wheels = w;
        seats = s;
    }
};

class Vehicle
{
    VehicleParts vp;

public:
    Vehicle(int w, int s) : vp(w, s)
    {
        cout << "Vehicle created\n";
    }
};
```

Nested Classes (Classes Inside Classes)

C++ allows defining a class inside another class. These are called nested classes.

A nested class is a normal class but its scope is limited inside the outer class.

If declared in public section, it can be accessed outside. If declared in private section, it can only be used inside the outer class.

Nested Class Example

```
class Outer
{
public:
    class Inner
    {
public:
```

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

```
void show()
{
    cout << "Inner class\n";
}
};
```

Usage:

```
Outer::Inner obj;
obj.show();
```

Important Concepts

Inner classes are only type definitions unless objects are created.

Outer class does not automatically contain inner class objects.

Friend classes can be used to allow access between inner and outer classes.

Tips

Objects inside classes are constructed before the outer object.

Destruction happens in reverse order.

Initializer lists should be used for member objects.

Nested classes help group logically related classes together.

Structures can also be used as class members.

Important Exam Points

Member objects are constructed before the outer class object.

Destructor execution order is reverse of constructor order.

Initializer list is required for initializing member objects.

Nested classes are classes defined inside another class.

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Structures and classes behave similarly in C++ except default access specifier.

5 MCQs (Important)

Q1. In what order are member objects constructed?

- A) After outer object
 - B) Before outer object ✓
 - C) Random order
 - D) After destructor
-

Q2. Which is used to initialize member objects?

- A) Constructor body
 - B) Initializer list ✓
 - C) Destructor
 - D) Loop
-

Q3. Destructor order is:

- A) Same as constructor
 - B) Reverse of constructor ✓
 - C) Random
 - D) Undefined
-

Q4. Default access in class is:

- A) Public
 - B) Private ✓
 - C) Protected
 - D) Static
-

Q5. Nested classes are:

- A) Functions
 - B) Classes inside classes ✓
 - C) Arrays
 - D) Objects only
-

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Lecture No. 41 – Template Functions in C++

Lecture Overview

In previous lectures, we discussed objects used as data members inside classes and how code reuse is achieved using object composition.

In this lecture, we introduce a new and powerful way of code reuse in C++ called **Templates**.

Templates allow us to write **generic code** that works with multiple data types without rewriting the same function again and again.

Template Functions

Templates are a feature of C++ that allow writing **generic functions and classes**.

There are two types of templates:

Function Templates

Class Templates

A function template allows a single function definition to work with different data types automatically.

A template works like a **stencil or pattern** used to create multiple outputs from one design idea.

Need for Template Functions

Consider a swap function that exchanges two integers using a temporary variable.

If we want to perform the same operation for different data types like int, float, and double, we need separate functions for each type.

This causes **code repetition and poor reuse**.

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Templates solve this problem by allowing a single function to work with all data types.

Template Function Syntax

```
template  
return_type function_name(parameters)
```

Here, template is a keyword and class T defines a generic data type.

T acts as a placeholder and is replaced by actual data types when the function is used.

Example: Template Reverse Function

Normal function for integer returns negative value of x.

```
int reverse(int x)  
return -x
```

Double version does the same for double type.

Using templates, we write only one function:

```
template  
T reverse(T x)  
return -x
```

This single function works for int, float, double and char.

How Templates Work

When a template function is called, the compiler detects the data type automatically.

It replaces T with the actual data type and generates a specific version of the function at compile time.

For example, reverse(5) creates an integer version and reverse(3.5) creates a double version.

This process happens during compilation, not at runtime.

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Template Swap Function Example

```
template  
void swap(T &x, T &y)  
{  
    T temp  
    temp = x  
    x = y  
    y = temp  
}
```

This single function can swap values of int, float, char, and even objects.

Multiple Template Parameters

Templates can also use more than one generic type.

```
template<class T, class U>
```

This allows functions to handle mixed data types such as int and float together.

Template Function Overloading

Template functions can be overloaded like normal functions.

Functions with the same name can exist if they differ in number or type of parameters.

The compiler selects the correct function automatically based on arguments.

Explicit Type Specification in Templates

Sometimes we force the compiler to use a specific data type.

`reverse(a)` forces the function to behave as an int version.

We can also specify multiple types such as `reverse<int, double>(a)`.

This allows control over template instantiation.

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Templates with Objects

Template functions also work with user-defined classes.

However, the class must support all operations used inside the template function.

For example, if the template uses `-x`, the class must overload the minus operator.

Example: PhoneCall with Template

A class PhoneCall contains call duration and bill code.

Unary minus operator is overloaded to mark a call as cancelled by changing bill code to 'c'.

When reverse function is used with PhoneCall object, the operator is triggered automatically and the object is modified.

Advantages of Template Functions

Templates reduce code duplication and improve reusability.

They allow one function to work with multiple data types.

They improve maintainability and support generic programming.

Limitations of Templates

Template code must be written in the same file.

All required operations must be defined in the class.

Errors appear at compile time and can be harder to debug.

Key Points About Templates

Templates are resolved at compile time.

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

They automatically generate type-specific functions.

At least one generic parameter is required.

They are more powerful than function overloading in terms of reuse.

Important Exam Points

Definition and syntax of template functions is important.

Difference between templates and function overloading should be clear.

Explain compile-time code generation process.

Explain explicit type specification in templates.

Explain use of templates with user-defined classes.

Explain advantages and limitations of templates.

MCQs – Template Functions in C++

Q1: What is a template in C++?

- A) A loop structure
 - B) A function that works only for integers
 - C) A generic code pattern used for multiple data types ✓
 - D) A memory management tool
-

Q2: When are template functions instantiated?

- A) Runtime
 - B) Compile time ✓
 - C) Link time
 - D) Execution end
-

Q3: What does template<class T> mean?

- A) T is a fixed data type
- B) T is a pointer

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

- C) T is a generic placeholder for any data type ✓
D) T is a class name
-

Q4: Which is required for a template function?

- A) No parameters
B) At least one generic type parameter ✓
C) Only integer parameters
D) Only return type must be generic
-

Q5: What happens when a template is used with a class object?

- A) It only works for built-in types
B) Compiler deletes the function
C) Compiler generates a version if required operations are defined ✓
D) It always causes runtime error
-

Lecture No. 42 – Class Templates in C++

Lecture Overview

In previous lectures, we studied template functions and how they allow code reuse for different data types.

In this lecture, we extend the concept of templates to **class templates**, which allow us to create **generic classes** that can work with any data type.

We also study advanced concepts such as non-type parameters, static members, friend functions, and an introduction to the **Standard Template Library (STL)**.

Class Templates

Class templates allow us to define a **generic class blueprint** that can work with different data types.

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Instead of writing separate classes for int, float, or double, we write one generic class using a placeholder type.

The syntax is:

```
template  
class ClassName  
{  
    // class definition using T  
};
```

Here, T is a **generic data type** that is replaced at compile time.

Need for Class Templates

If we want to create a stack for integers, floats, and doubles, we normally need separate classes.

This leads to **code duplication and poor reuse**.

Class templates solve this problem by allowing a single class definition to work for all data types.

Example: Generic Class

A simple template class example is:

```
template  
class Number  
{  
    T value;  
public:  
    Number(T v);  
    void display();  
};
```

We can create objects as:

```
Number a;  
Number b;
```

The compiler automatically generates separate versions of the class for each data type.

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

How Class Templates Work

When an object is created:

The compiler replaces T with the actual data type

It generates a complete class version for that type

Each version behaves like a normal class

This process happens at **compile time**

Example:

Number → creates integer version

Number → creates double version

Non-Type Parameters in Templates

Templates can also use **constant values as parameters**.

Syntax:

```
template <class T, int size>
```

Here, size is a **non-type parameter**.

It behaves like a constant and is used for fixed-size structures like arrays.

Static Members in Class Templates

Static members behave differently in template classes.

Each template instantiation creates a **separate static member copy**.

Example:

Number → one static copy for int version

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Number → separate static copy for double version

So static members are **type-specific, not class-wide across all types**.

Friend Functions in Templates

Friend functions allow external functions to access private data of a class.

In templates:

A friend function can be shared across all template instantiations

Or it can be made type-specific using T

Example:

```
friend void show();
```

This function becomes friend to all versions of the template class.

Template Class with Stack Example

A stack is a good example of a class template because it follows LIFO (Last In First Out).

```
template  
class Stack  
{  
    T arr[SIZE];  
    int top;  
public:  
    void push(T);  
    T pop();  
    bool isEmpty();  
    bool isFull();  
};
```

This stack can now work for:

```
int stack
```

```
float stack
```

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

double stack

char stack

Templates and Objects

Template classes can also be used with **user-defined types (classes)**.

Example:

Stack s;

This is valid if the class supports required operations.

Advantages of Class Templates

Class templates reduce code duplication.

They improve code reusability and maintainability.

They ensure type safety at compile time.

They allow creation of generic data structures.

Disadvantages of Templates

Template code can be difficult to debug.

Errors are often complex and hard to understand.

Code must be written in the same file.

Large template code may reduce readability.

Standard Template Library (STL)

STL is a collection of **predefined template classes and functions** in C++.

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

It provides tested and optimized implementations of:

Vectors

Stacks

Queues

Lists

Algorithms

STL promotes **maximum code reuse and reliability**.

Instead of writing custom data structures, programmers can use STL components directly.

Key Points About Class Templates

Class templates are used to define generic classes.

They are instantiated at compile time.

Each data type generates a separate class version.

Static members are type-specific in templates.

Templates support both built-in and user-defined types.

Important Exam Points

Definition and syntax of class templates

Difference between template functions and class templates

Concept of non-type parameters

Behavior of static members in templates

Use of templates with user-defined classes

Advantages and disadvantages of templates

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Basic idea of STL and its importance

MCQs

Q1: What is a class template in C++?

- A) A loop structure
 - ✓ B) A generic class definition for multiple data types
 - C) A pointer type
 - D) A runtime function
-

Q2: When are class templates instantiated?

- A) Runtime
 - ✓ B) Compile time
 - C) Execution time
 - D) Link time
-

Q3: What is T in template?

- A) A fixed data type
 - B) A variable name
 - ✓ C) A generic placeholder type
 - D) A class object
-

Q4: What happens to static members in template classes?

- A) One shared copy for all types
 - B) No static members allowed
 - ✓ C) Separate copy for each data type
 - D) Deleted by compiler
-

Q5: What is STL in C++?

- A) A compiler tool
 - B) A debugging system
 - ✓ C) Standard Template Library
 - D) A memory manager
-

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Lecture No. 43 – Programming Exercise: Matrices in C++

Lecture Overview

In this lecture, we apply object-oriented programming and class design concepts to a real-world mathematical problem: **Matrices**.

We use the **design recipe approach** which includes problem analysis, class design, implementation planning, and testing.

The goal is to design a flexible **Matrix class** that can perform standard matrix operations efficiently.

Design Recipe

The design recipe is a structured approach to solving programming problems.

First, we analyze the problem and clearly define requirements.

Next, we design data structures and decide how the class will represent the problem.

Then we implement the solution in C++.

Finally, we test the program with different inputs and fix errors if found.

This cycle continues until the program works correctly in all cases.

Problem Analysis

A matrix is a **two-dimensional array of numbers** arranged in rows and columns.

A matrix of size $m \times n$ has m rows and n columns.

In programming, matrices are widely used in graphics, simulations, and scientific computing.

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Matrix Operations

Several operations can be performed on matrices.

Matrix addition requires two matrices of the same order and adds corresponding elements.

Scalar addition adds a single number to every element of the matrix.

Matrix subtraction subtracts corresponding elements of two matrices of the same size.

Scalar subtraction subtracts a number from each element of a matrix.

Scalar multiplication multiplies every element of the matrix by a constant value.

Matrix multiplication requires the number of columns of the first matrix to equal the number of rows of the second matrix.

Transpose of a matrix is obtained by swapping rows with columns.

Matrix Addition and Subtraction

In matrix addition, if A and B are matrices of the same order, then each element is added as $A_{ij} + B_{ij}$.

The resulting matrix has the same dimensions.

Matrix subtraction works similarly as $C_{ij} = A_{ij} - B_{ij}$.

Both matrices must have identical dimensions for these operations to be valid.

Scalar Operations

In scalar operations, a single number is applied to all elements of the matrix.

For scalar multiplication, each element is multiplied by the scalar value.

For scalar division, each element is divided by the scalar, provided the scalar is not zero.

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Matrix Multiplication

Matrix multiplication is more complex than other operations.

The number of columns in the first matrix must equal the number of rows in the second matrix.

Each element of the result is computed by multiplying corresponding row and column elements and summing them.

The resulting matrix has dimensions $m \times p$ if the first matrix is $m \times n$ and the second is $n \times p$.

Transpose of a Matrix

Transpose is obtained by interchanging rows and columns.

Each row of the original matrix becomes a column in the new matrix.

For square matrices, dimensions remain the same.

For non-square matrices, rows and columns are swapped.

Design Issues and Class Interface

The Matrix class should support dynamic memory allocation since matrix size is not fixed.

The size is provided at runtime through rows and columns.

A constructor is used to allocate memory for the matrix.

A display function is used to print matrix contents.

Matrix Class Interface

The constructor takes number of rows and columns as input.

Matrix(int rows, int cols);

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Operator overloading is used to implement matrix operations such as $+$, $-$, $*$, and $/$.

These operators allow natural mathematical syntax in code.

Operator Overloading in Matrix Class

Matrix addition is implemented using operator $+$ as a member function.

Matrix subtraction is implemented using operator $-$.

Scalar operations are implemented using overloaded operators that accept numbers instead of matrices.

Friend Functions in Matrix Operations

Some operations require friend functions because the scalar appears on the left side.

For example, $x + A$ cannot be handled by a member function.

So a friend function is used for such cases.

Friend functions allow external access to private data when needed.

Transpose Function

The transpose function converts rows into columns.

It returns a reference to the modified matrix.

Matrix & transpose();

This allows efficient manipulation without unnecessary copying.

Memory Management in Matrix Class

Since dynamic memory is used, proper cleanup is required.

A destructor is used to free allocated memory.

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

A copy constructor is required for deep copying.

The assignment operator is overloaded to prevent shallow copy issues.

Important Design Considerations

Matrix operations must check dimension compatibility.

Division must ensure scalar is not zero.

Memory must be carefully managed to avoid leaks.

Operator overloading improves readability and usability.

Template Extension Idea

The Matrix class can be converted into a template class.

This allows it to support int, float, and double types.

Scalar values can also be made generic using templates.

This improves reusability and flexibility.

Alternative Design Approach

Instead of a single Matrix class, a Row class can be used.

Multiple Row objects can be combined to form a Matrix.

However, a single-class design is simpler and easier for learning purposes.

Key Points About Matrix Class Design

Matrix is a 2D dynamic data structure.

Operator overloading simplifies mathematical operations.

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Friend functions are required for scalar-first operations.

Deep copy is essential due to dynamic memory allocation.

Transpose and multiplication require careful logic.

Important Exam Points

Definition and structure of a matrix in programming

Rules of matrix addition, subtraction, multiplication, and transpose

Need for dynamic memory allocation in Matrix class

Role of constructor, destructor, copy constructor, and assignment operator

Use of operator overloading in matrix operations

Use of friend functions in scalar-matrix operations

Importance of deep copy in dynamic structures

MCQs

Q1: What is a matrix in programming?

- A) A one-dimensional array
- ✓ B) A two-dimensional array of numbers
- C) A pointer variable
- D) A linked list

Q2: Which condition is required for matrix addition?

- A) Different sizes allowed
- ✓ B) Same number of rows and columns
- C) Only square matrices
- D) Only integer matrices

Q3: What is transpose of a matrix?

- A) Multiplying all elements
- ✓ B) Interchanging rows and columns

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

- C) Inverting the matrix
 - D) Adding scalar value
-

Q4: Why are friend functions used in Matrix class?

- A) For faster execution
 - B) For memory allocation
 - ✓ C) For scalar-first operations like $x + A$
 - D) For deleting objects
-

Q5: Why is a copy constructor needed in Matrix class?

- A) For printing matrix
 - B) For input handling
 - ✓ C) For deep copying dynamic memory
 - D) For sorting elements
-

Lecture 44 – Matrix Class (Dynamic Memory, Constructors, Operators)

Lecture Overview

In this lecture, we review the implementation of a Matrix class in C++ with special focus on dynamic memory allocation, constructors, destructors, and operator overloading.

We revisit earlier matrix concepts and convert them into a complete reusable class using C++ features like new/delete, copy constructor, assignment operator, and stream handling.

The class is designed for double-type matrices and supports flexible sizes such as 2×2 , 4×5 , or 20×20 . It uses dynamic memory allocation so that memory is created at runtime based on required dimensions.

Matrix Class Design

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

The Matrix class stores numRows as number of rows, numCols as number of columns, and elements as a 2D dynamic array (double pointer).

The matrix is stored as double **elements.

This means the first pointer represents an array of row pointers and the second pointer represents actual row data (columns).

Each object can represent any size matrix depending on runtime input.

Key Concept: Dynamic Memory Allocation

Instead of fixed arrays, memory is allocated using new.

Rows are allocated first as a pointer array and then each row gets its own column array.

This allows flexible matrix sizes and avoids wastage of memory.

Important Requirement in This Class

Since dynamic memory is used, we must define a constructor that allocates memory, a destructor that releases memory, a copy constructor that performs deep copy, and an assignment operator that avoids shallow copy issues.

Without these, memory errors and leaks occur.

Constructors

1. Default Constructor

Used to create a matrix with given size or default 0×0.

It stores row and column size, allocates memory dynamically, and initializes all elements to 0.0.

2. Copy Constructor

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Used when Matrix $A = B$ or Matrix $A(B)$.

It performs a deep copy in which new memory is allocated and values are copied element by element.

This prevents shared memory problems.

Destructor

The destructor releases dynamically allocated memory and prevents memory leaks.

It uses delete [] elements to free allocated memory and ensures proper cleanup when the object goes out of scope.

Utility Functions

getRows() returns number of rows.

getCols() returns number of columns.

Both are constant functions because they do not modify the object.

Input Functions

1. Keyboard Input Function

This function takes input from the user using cin, displays prompts for each row, and accepts space-separated values where spaces act as delimiters.

2. File Input Function

This function reads a matrix from a file by first reading rows and columns and then reading all elements.

A temporary matrix is used and the assignment operator is applied for deep copy.

The matrix size is not known in advance, so it is stored in the file first.

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Output Functions

1. Screen Output Function

This function displays the matrix in a formatted box style using ASCII characters.

It uses setw for alignment, setprecision(2) for formatting, and special characters for borders to produce a graphical matrix layout.

2. File Output Function

This function writes the matrix to a file by storing number of rows, number of columns, and all elements.

No graphical formatting is used because files are meant for data storage.

Transpose Function

Transpose converts rows into columns and columns into rows.

For square matrices, it is done in-place using swapping of elements without extra memory.

For non-square matrices, a new matrix is created and elements are copied with swapped indices using $\text{temp}[j][i] = \text{original}[i][j]$, then assignment operator replaces the original matrix.

Operator Overloading

The class supports Matrix + Matrix, Matrix + double, Matrix - Matrix, Matrix - double, Matrix * Matrix, Matrix * double, double * Matrix, and Matrix / double.

Each operator returns a new Matrix.

Compound Assignment Operators

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

The operators += and -= modify the same object using previously defined operators.

Example: $A += B$ means $A = A + B$.

Assignment Operator (=)

The assignment operator handles deep copy, reallocates memory if size changes, and prevents shallow copy issues.

Friend Operators

Friend operators are used when the left operand is not a Matrix, such as double + Matrix, double - Matrix, or double * Matrix.

Stream Operators

The operators >> and << are used for input and output.

They support both keyboard and file operations and allow the Matrix class to behave like built-in data types.

Key Concepts from Main Function

The program demonstrates matrix creation, input/output handling, transpose operation, arithmetic operations, operator overloading, file saving and loading, and compound expressions such as $m = m + m + 4$, $m += n$, and $c = m * n$.

Important Exam Points

Matrix uses dynamic memory allocation with double pointer.

Constructor allocates memory while destructor frees it.

Copy constructor performs deep copy.

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Assignment operator prevents shallow copy issues.

Transpose swaps rows and columns.

Operator overloading allows matrix operations like built-in types.

Friend functions are required when left operand is not a class object.

Stream operators enable file and console input/output.

File storage includes matrix dimensions and elements.

Nested loops are essential for matrix processing.

MCQs

1. Why is a destructor required in the Matrix class?

- A) To increase speed of program
- B) To allocate memory
- C) To free dynamically allocated memory ✓
- D) To print matrix

2. What does the copy constructor do in the Matrix class?

- A) Performs shallow copy
- B) Performs deep copy ✓
- C) Deletes memory
- D) Initializes only rows

3. In transpose of a non-square matrix, what happens?

- A) Matrix is deleted
- B) Matrix is reversed
- C) New matrix is created with swapped dimensions ✓
- D) Only diagonal is changed

4. Why are friend functions used in operator overloading?

- A) To reduce memory usage
- B) When left operand is not a class object ✓
- C) To avoid constructors
- D) To delete objects

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

5. What is stored first in the file representation of a matrix?

- A) Matrix elements
- B) Determinant
- C) Number of rows and columns ✓
- D) Only columns

Lecture 45 – Matrix Class (Operator Overloading, Programming Concepts Review)

Lecture Overview

This lecture continues the discussion of the **Matrix class in C++**, focusing on the implementation of key **operator overloading functions**, especially assignment, arithmetic, and stream operators.

It also includes a **comprehensive review of programming fundamentals**, such as variables, loops, arrays, classes, pointers, memory management, and SQL concepts.

The lecture strengthens understanding of how **object-oriented programming and dynamic memory work together in real applications**.

Example (Continued – Matrix Class)

This lecture continues the previously developed **Matrix class**, which uses dynamic memory allocation. The focus is on improving functionality through operator overloading and ensuring safe memory handling.

The matrix class supports flexible sizes and uses **double pointer (double **)** **representation** for storing elements dynamically.

Assignment Operator Function

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

The assignment operator is one of the most critical functions in the Matrix class because it handles **dynamic memory safely**.

When writing expressions like:

$a = b$

both matrices may have different sizes. Therefore, the assignment operator must:

Check for self-assignment using `if(&m != this)`

If matrices have different sizes, deallocate existing memory and reallocate new memory based on the right-hand side matrix

Copy elements one by one using nested loops

Update number of rows and columns after reallocation

Return `*this` as a constant reference to allow chaining like `a = b = c`

The operator ensures **deep copy behavior**, preventing memory corruption and leaks.

Addition Operator Function

The `+` operator performs matrix addition only when both matrices are **conformable (same dimensions)**.

A temporary matrix is created using the copy constructor. Then corresponding elements are added using nested loops.

If matrices are not compatible, the original matrix is returned unchanged.

The function returns a **new matrix object**, ensuring original matrices remain unchanged.

Plus-Equal Operator (`+=`)

The `+=` operator modifies the calling object directly.

It uses the existing `+` operator internally and assigns the result back to the same object.

The function returns a reference to the current object (`*this`), enabling chaining.

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

This operator improves efficiency because it avoids unnecessary temporary objects.

Overloaded Plus Operator with Double

This version allows adding a scalar value to every element of the matrix.

A temporary copy of the matrix is created using the copy constructor.

Each element is increased by the given double value using nested loops.

A new modified matrix is returned while the original remains unchanged.

Friend Addition Operator (Double + Matrix)

When the left operand is not a matrix (for example, double + Matrix), a friend function is used.

Since the left operand is not a class object, the operator is defined outside the class.

The function creates a temporary matrix, adds the scalar to each element, and returns the result.

This ensures **commutative behavior for scalar addition**.

Minus Operator Function

The minus operator works similarly to addition but follows non-commutative rules.

Matrix subtraction requires conformability (same dimensions).

Element-wise subtraction is performed using nested loops.

For scalar subtraction, the result differs depending on order ($a - d \neq d - a$).

Multiplication Operator Function

Matrix multiplication is the most complex arithmetic operation.

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

The result matrix size is:

rows of first matrix $\times$ columns of second matrix

Multiplication requires checking:

Number of columns of first matrix = number of rows of second matrix

Each element is calculated using three nested loops:

Row $\times$ Column multiplication and summation

If matrices are not conformable, an empty or default matrix is returned.

Division by Scalar

Matrix division by a double is simple.

Each element is divided by the scalar value.

A check is performed for division by zero.

If zero is detected, the original matrix is returned without modification.

This avoids runtime crashes but is logically not the best approach compared to exception handling.

Stream Insertion and Extraction Operators

Stream operators allow matrices to work with input/output streams.

The extraction operator ($>>$) reads matrix input and internally calls the input function.

The insertion operator ($<<$) displays the matrix using the output function.

Both file and console versions are implemented using **function overloading**.

These operators demonstrate **code reuse and clean interface design**.

Exercise

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Students are encouraged to:

Create a Matrix object using Matrix m(3,3)

Display it using output functions

Perform addition, multiplication, and scalar operations

Test conformability rules

Extend the class with additional features like error handling

Convert the class from double to int or template-based implementation

Improve the program by adding proper error messages and validations

Rules for Programming

Only three constructs are required to solve any programming problem.

The first is sequence where instructions are executed step by step.

The second is decision making using if, if-else, or switch statements.

The third is looping structures that repeat tasks using while, do-while, or for loops.

Good programming requires clean structure, proper indentation for readability, meaningful comments, and avoidance of unnecessary complexity.

Variables and Pointers

A variable represents a memory location with a symbolic name.

A pointer stores the address of another memory location.

Pointers allow direct memory manipulation but must be used carefully due to risks like dangling pointers or memory leaks.

Arrays

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Arrays store multiple elements of the same data type.

They provide indexed access to elements.

Different languages support different array behaviors, including fixed-type and variant-type arrays.

Loops and Decisions

Decision structures include if, if-else, and switch statements.

Loop structures include while, do-while, and for loops.

While loops execute zero or more times, do-while executes at least once, and for loops are compact forms of iteration.

Classes and Objects

Classes combine data and functions into a single unit.

Objects are instances of classes.

This supports encapsulation, data hiding, and reusability.

Classes make large programs manageable by separating interface from implementation.

Garbage Collection

C++ requires manual memory management using new and delete.

Incorrect handling leads to memory leaks or dangling pointers.

Modern languages like Java use garbage collection to automatically free unused memory.

Truth Table

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

Truth tables help evaluate logical expressions systematically.

They are useful in simplifying Boolean logic and designing circuits.

They ensure clarity in complex decision-making conditions in programming.

Structured Query Language (SQL)

SQL is a declarative language used for database operations.

Unlike procedural languages, SQL focuses on what is required rather than how to achieve it.

Database systems use optimizers to execute queries efficiently.

SQL is widely used in enterprise applications and data management systems.

Important Exam Points

Matrix class uses dynamic memory with double pointer representation

Assignment operator must handle self-assignment and deep copy

Arithmetic operators return new objects except += which modifies current object

Multiplication requires strict conformability rules

Stream operators enable clean input/output handling

Pointers require careful memory management to avoid leaks

Loops, decisions, and sequences form the foundation of programming

Classes provide encapsulation and code reusability

SQL is a declarative language unlike procedural C++

Proper operator overloading improves usability of user-defined types

MCQs

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

1. What is the main purpose of the assignment operator in the Matrix class?

- A. To print matrix
 - B. To handle file input
 - C. To perform deep copy and manage dynamic memory (✓)
 - D. To delete matrix elements
-

2. Why do we check self-assignment in the assignment operator?

- A. To improve speed
 - B. To avoid memory deletion of same object (✓)
 - C. To increase matrix size
 - D. To print error message
-

3. What does the += operator return?

- A. A new matrix
 - B. A pointer
 - C. Reference to the same object (✓)
 - D. Integer value
-

4. What is required for matrix multiplication?

- A. Same number of rows
 - B. Same number of columns
 - C. Column of first equals row of second (✓)
 - D. Equal total elements
-

5. SQL is best described as a:

- A. Procedural language
 - B. Low-level language
 - C. Declarative language (✓)
 - D. Assembly language
-

■ ■■■■ | ■■ | ■■ ■■ ■■ THE END ■ | ■■ ■■ | ■■■ ■■ ■■

CS201 IMPORTANT NOTES + EXAM KEY POINTS + LECTURE WISE MCQS

VU MASTERY: YOUR ACADEMIC EDGE



VU MASTERY: YOUR ACADEMIC EDGE



Providing Expert LMS Handling Service |
Achieving 100% Guaranteed Results for
Virtual University Students.

UNLOCK YOUR FULL POTENTIAL.



COMPLETE LMS SOLUTIONS

- ✓ GDB (Graded Discussion Board)
- ✓ Assignments
- ✓ Quizzes
- ✓ Files & Projects
- ✓ All Subjects Covered

WHY CHOOSE US?

- **Unmatched Expertise:**
Decades of combined experience.
- **Confidentiality & Privacy:**
Secure and private service.

EXCELLENCE GUARANTEED

100%

We are committed to
delivering **100% Guaranteed
Results.**
Achieve your desired grades
with our support.

- ✓ **Time Management:**
Free up your schedule.
- ✓ **Stress Reduction:**
Let us handle the pressure.
- ✓ **Consistent Updates:**
Track your progress.

GET STARTED TODAY

**For More Information,
Contact Us Now:**

Contact Us

WhatsApp +92 370 5482939

WhatsApp +92 370 5482939

www.vumastery.com

**Don't let LMS stress hold
you back. Elevate your
studies with VU Mastery.**